

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

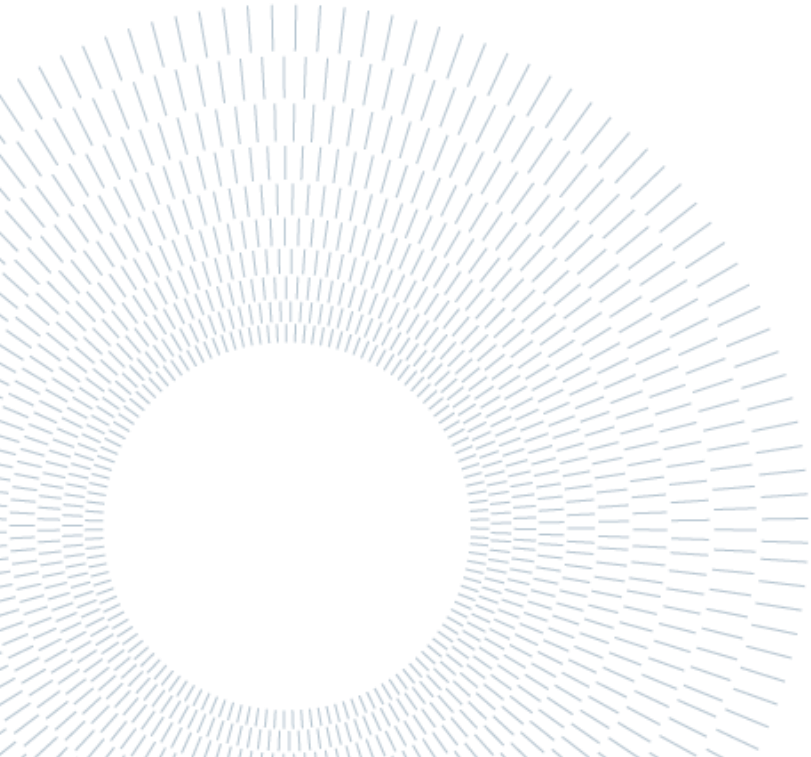
DD

Design Document

Best Bike Paths

Author: **Anna Cordioli – Tasnem El Koush**

Students: Anna Cordioli – 10891104
Tasnem El Koush - 10807328
Professor: Elisabetta Di Nitto
Academic Year: 2025-26



Contents

List of Figures v
List of Tables viii
1. Introduction 9
1.1. Purpose 9
1.2. Scope 9
1.3. Definitions, Acronyms, Abbreviations 9
1.4. Revision history 10
1.5. Reference Documents 10
1.6. Document Structure 10
2. Architectural Design 12
2.1. Overview: High-level components and their interactions 12
2.2. Component View 13
2.2.1. High-level components 13
2.2.2. Low-level components 15
2.2.3. Dashboard Manager 15
2.2.4. Authentication Manager 16
2.2.5. Profile Manager 17
2.2.6. Searching Manager 17
2.2.7. Manual Insertion Manager 18
2.2.8. Collector Manager 18
2.2.9. Notification Manager 19
2.2.10. Weather Integration Manager 19
2.2.11. Database Query Manager 19
2.3. Deployment View 20
2.4. Runtime View 22
2.5. Component Interfaces 33
2.5.1. Authentication Manager 33
2.5.2. Collector Manager 33
2.5.3. Manual Insertion Manager 34
2.5.4. Searching Manager 36
2.5.5. Sensor Data Collector 36
2.5.6. Weather Integration Manager 36
2.5.7. Database Query Manager 36
2.5.8. Notification Manager 37
2.5.9. External Service Interfaces 37

2.6. Selected architectural styles and patterns 38
2.6.1. Architectural Design 38
2.6.2. Client-Server Communication 39
2.6.3. Service-Oriented Design Pattern 39
2.6.4. Repository Pattern (Data Access) 39
2.6.5. Offline-First Pattern 40
2.7. Other design decisions 40
2.7.1. Auxiliary Components 40
2.7.2. Other integrated aspects 41
3. User interface design 43
3.1. Welcome and Authentication Flows 43
3.1.1. Welcome Screen and Access Options 43
3.1.2. Registration Flow 44
3.1.3. Login Flow 48
3.2. Guest User Interface 50
3.2.1. Search Bike Paths 50
3.2.2. Map Visualization 52
3.2.3. Path Details 53
3.3. Registered User Interface 55
3.3.1. Homepage and Dashboard 55
3.3.2. Recording Trip Flow 56
3.3.3. Manual Path Insertion Flow 60
3.3.4. Automated Trip Detection Flow 62
3.3.5. Search Bike Paths 63
3.3.6. My Trips and Weather Enrichment 64
3.3.7. User Profile and Settings 67
4. Requirements traceability 68
5. Implementation, integration and test plan 70
5.1. Overview 70
5.2. Features Identification 70
5.3. Integration Strategy 72
5.4. System Testing Strategy 77
6. Effort spent 81
7. References 82
7.1. References 82
7.2. Used Tools 82

List of Figures

Figure 1 - High level components diagram 12
Figure 2 - High level components view 13
Figure 3 - Low level diagram 15
Figure 4 - Authentication Manager Component 16
Figure 5 - Profile Manager component 17
Figure 6 - Searching Manager component 17
Figure 7 – Manual Insertion Manger component 18
Figure 8 - Collector Manager component 18
Figure 9 - Deployment diagram 20
Figure 10 - Login sequence diagram 22
Figure 11 - Registration sequence diagram 23
Figure 12 - Manual trip recording sequence diagram 24
Figure 13 - Searching sequence diagram 25
Figure 14 - My trips sequence diagram 26
Figure 15 - Automated detection sequence diagram 27
Figure 16 – Manual path insertion sequence diagram 29
Figure 17 – Review, save and publication options sequence diagram 30
Figure 18 - Weather enrichment sequence diagram 32
Figure 19 - User access options flow 43
Figure 20 - Welcome screen 43
Figure 21- User registration flow 44
Figure 22 - Account creation screen 44
Figure 23 - Email verification prompt 45
Figure 24 - Registration successful confirmation 46
Figure 25 – Permission configuration screen 47
Figure 26 – Registered User login flow 48
Figure 27 - Login screen 48
Figure 28 - Registered user homepage 49

Figure 29- Guest user search flow 50
Figure 30 - Guest mode homepage and search interface 50
Figure 31 - Ranked search results for guest users 51
Figure 32 - Map visualization 52
Figure 33 – Path details view 53
Figure 34 - Expanded map view with street sequence 54
Figure 35 - Registered user homepage 55
Figure 36 – Trip recording flow 56
Figure 37 - Active trip recording screen 56
Figure 38 - Trip data and performance metrics data 57
Figure 39 - Review Trip (upper section) 58
Figure 40 - Review Trip (lower section) 58
Figure 41 – Save and publication options screen 59
Figure 42 – Manual path insertion flow 60
Figure 43- Route definition (upper section) 60
Figure 44 - Route definition (lower section) 60
Figure 45- Trip details for manual path insertion 61
Figure 46- Save and publish options for manual path insertion 61
Figure 47 – Automated trip detection flow 62
Figure 48 – Notification screen 62
Figure 49 – Registered user search flow 63
Figure 50 - Search interface 63
Figure 51 - Ranked search results 63
Figure 52 - Map visualization 63
Figure 53 - My Trips access flow 64
Figure 54 - Trip history with filtering options 64
Figure 55 - Trip details with weather enrichment 65
Figure 56 - Private trip with publish option 66
Figure 57 - User profile 67
Figure 58 - Settings 67
Figure 59 - Edit profile 67

Figure 60 - Integration Strategy, Step 0 73
Figure 61 - Integration Strategy, Step 1 73
Figure 62 - Integration Strategy, Step 2 74
Figure 63 - Integration Strategy, Step 3 74
Figure 64 - Integration Strategy, Step 4 75
Figure 65 - Collector Manager - Internal Sub-components Testing 75
Figure 66 - Integration Strategy, Step 5 76
Figure 67 - Integration Strategy, Step 6 76
Figure 68 - Integration Strategy, Step 7 77

List of Tables

Table 1 - Acronyms definition 10

1. Introduction

1.1. Purpose

The purpose of this Design Document is to describe the architectural and design choices adopted for the development of the *Best Bike Paths (BBP)* system. This document provides a detailed overview of the system's structure, components, and interactions, serving as a reference for developers, designers, and stakeholders involved in the implementation and maintenance of the application.

This document aims to clarify how the system fulfils the functional and non-functional requirements specified for BBP, including data acquisition, validation, storage, and visualization, as well as user interaction and integration with external services.

1.2. Scope

As described in the Requirements Analysis and Specification Document (RASD), BBP is an application that aims to improve cycling experience by providing information about bike paths and supporting users in searching for and recording their trips to track their performance.

The application allows all users to search for routes between a starting point and a destination, view them on a map, and obtain information on their status, the street names that compose the path, and any reported obstacles. RU can contribute to the platform by entering or collecting bike path information, either manually or through device sensor tracking. The information can be saved as personal activities and also shared as a public bike path with the community.

By facilitating the sharing of paths within the cycling community, BBP aims to provide useful information for choosing suitable routes and to promote collaboration within the cycling community.

1.3. Definitions, Acronyms, Abbreviations

Acronym	**Definition**
ACID	Atomicity Consistency Isolation Durability
API	Application Programming Interface
BBP	Best Bike Path

DBMS	Database Management System
DD	Design Document
GU	Guest User
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JWT	JSON Web Token
LTS	Long Term Support
RASD	Requirements Analysis Specification Document
REST	REpresentational State Transfer
RU	Registered User
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TCP	Transmission Control Protocol
U	Any user

Table 1 - Acronyms definition

1.4. Revision history

- Version 1.0: Initial Release →07/01/2026

1.5. Reference Documents

- Slide and lesson notes
- Assignment RDD AY 2025-2026
- Peer review feedback

1.6. Document Structure

1. **Introduction**
   This chapter introduces the Design Document for the *Best Bike Paths (BBP)* system and provides the context needed to understand its content. It outlines the purpose and scope of the document, clarifying its role in describing the design and architectural choices of the system. The chapter also defines the terminology, acronyms, and abbreviations used throughout the document to ensure a common understanding among readers. In addition, it reports the

document revision history and the reference materials on which the design is based, and it presents the overall structure of the document to guide the reader through the subsequent chapters.

2. **Architectural Desing**
   This chapter describes the architectural design of the *Best Bike Paths (BBP)* system, presenting its overall structure and organization. It provides a high-level overview of the main software components and their interactions, followed by more detailed views that illustrate the system from different perspectives, including component, deployment, and runtime views. The chapter also defines the interfaces exposed by the components and discusses the architectural styles, patterns, and key design decisions adopted, motivating the chosen solutions and highlighting their impact on system quality attributes.
3. **User Interface Design**
   This chapter focuses on the design of the user interface of the BBP system. It describes how users interact with the application, illustrating the main screens, navigation flows, and interaction mechanisms. The chapter highlights the design principles adopted to ensure usability, clarity, and accessibility for both registered and unregistered users.
4. **Requirements Traceability**
   This chapter addresses the traceability between system requirements and design solutions. It explains how the architectural and design elements of the BBP system map to the identified requirements, ensuring that all functional and non-functional requirements are adequately covered and that design choices can be traced back to their original motivations.
5. **Implementation, Integration, and Test Plan**
   This chapter outlines the approach for implementing, integrating, and testing the BBP system. It describes the planned development process, integration strategy among system components, and the testing activities aimed at verifying correctness, reliability, and overall system quality throughout the development lifecycle.
6. **Effort spent**
   This part summarizes the distribution of workload within the team, specifying the number of hours contributed by each member in preparing this document and performing related activities.
7. **Reference**
   This final section lists all external materials, technical papers, standards, and online resources referenced throughout the document.

2. Architectural Design

2.1. Overview: High-level components and their interactions

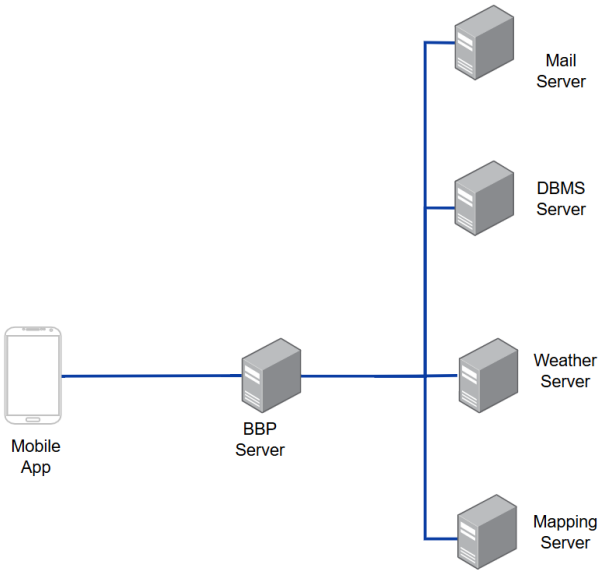


Figure 1 - High level components diagram

Client Side

- The **Mobile App** serves as the primary client interface for the Best Bike Paths (BBP) system. It enables users to record cycling trips with GPS tracking, collect accelerometer and gyroscope data for obstacles detection, manually input bike path information, and query for routes between origin and destination points. The app communicates with the BBP Server to store trip data, publish path information, and retrieve available route information with visualizations on an integrated map.

Server Side

- **BBP Server**: The core application server that handles all business logic for the BBP system. It processes trip recordings, manages bike path data (both manual and automated inputs), calculates path scores based on status and effectiveness, and orchestrates communication between the mobile app and backend services.
- **Mail Server**: Manages email notifications and communications with users for account verification.
- **DBMS Server**: The database management system that stores all persistent data including user accounts, trip records, bike path information, obstacle reports, and path ratings.

- **Weather Server**: Accessed through platform-native SDKs (iOS Weather API for iOS, Android Weather API for Android) that communicate with external weather services to provide meteorological data.
- **Mapping Server**: Integrated through platform-native SDKs (Apple Maps SDK for iOS, Google Maps SDK for Android) to provide geospatial services for route visualization, GPS coordinate processing, and map rendering to display bike paths.

2.2. Component View

2.2.1. High-level components

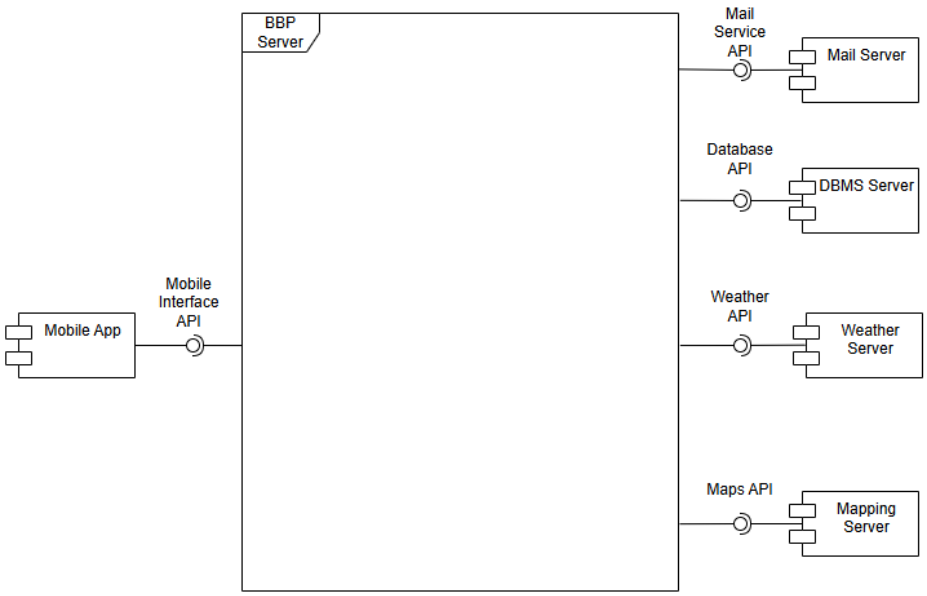


Figure 2 - High level components view

Mobile App: This is the mobile application that users interact with to access the BBP system. It communicates with the BBP Server through the *Mobile Interface API*, which is the primary means of interaction between users and the system. The mobile interface includes multiple functionalities: recording trips with automatic GPS tracking, collecting sensor data (accelerometer and gyroscope) for obstacle detection, manually inserting bike path information, querying routes between origin and destination points, and visualizing paths on an integrated map. The app can operate in both online and offline modes, synchronizing data with the server when connectivity is available.

BBP Server: This is the core application server that orchestrates all business logic and coordinates communication between the mobile app and backend services through dedicated APIs. It processes trip recordings, validates and stores bike path information (both manual and automated inputs), calculates path scores based on status and route

effectiveness, manages user authentication and authorization, handles route queries, and coordinates data enrichment from external services. The server exposes four main APIs to communicate with backend services: the *Database API* for data persistence operations, the *Mail Service API* for notification management, the *Weather API* for meteorological data retrieval, and the *Maps API* for geospatial services. It serves as the central hub connecting all system components.

DBMS Server: This server stores all the information related to users, including their profile information, trip history, recorded bike paths, obstacle reports, path ratings, and statistics. It is accessed by the BBP Server through the *Database API*, which handles all data management operations, ensuring data integrity and supporting complex queries for path scoring and route optimization. It is the main component for data persistence in the system.

Mail Server: This server is responsible for notifying users outside of the mobile application. It is accessed through the *Mail Service API* and is used to send notification for password recovery and registration confirm.

Weather Server: This component provides meteorological data to enrich trip information. It is accessed through platform-native *Weather APIs* and supplies real-time and historical weather conditions, temperature, wind speed, and precipitation data that are associated with recorded trips to provide users with comprehensive performance analytics and context for their cycling activities.

Mapping Server: This server is responsible for all geospatial functionalities within the system. It is accessed through platform-native *Maps APIs* and processes GPS coordinates to reconstruct followed paths, calculates optimal routes between origin and destination points, renders map visualizations with path overlays, computes distances and route effectiveness scores, and manages the geographic database of streets and bike-friendly routes. It periodically updates its data to reflect new paths and status changes reported by users.

2.2.2. Low-level components

The figure below shows a diagram of the low-level components that will be detailed in this section.

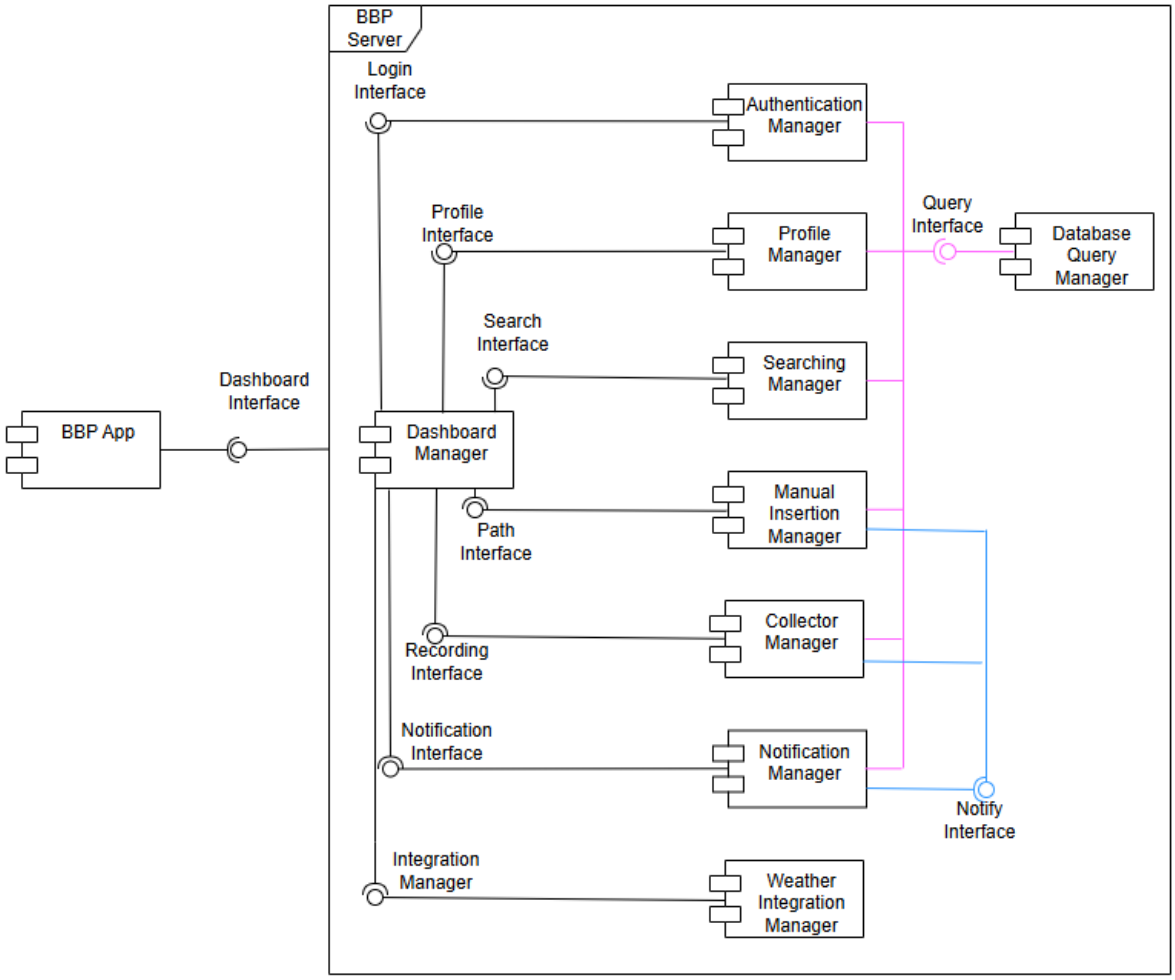


Figure 3 - Low level diagram

2.2.3. Dashboard Manager

This component coordinates user interactions through the mobile interface. It routes user requests to the appropriate internal components, manages the application navigation flow between different features (registration/login, recording, path insertion, search, trip history, profile management), ensures proper synchronization between the app and server components. The Dashboard Manager acts as the orchestration layer that connects user actions to backend functionality.

2.2.4. Authentication Manager

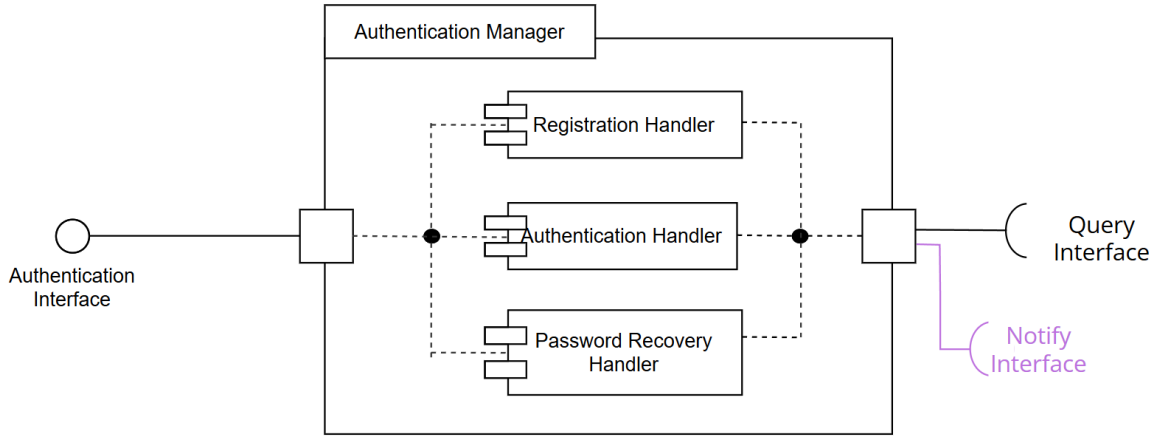


Figure 4 - Authentication Manager Component

This component is responsible for all user authentication and account management operations. It is composed of three specialized sub-components:

- **Registration Handler**: This component manages the user registration process. It validates registration data including email format and password, checks email uniqueness in the database to prevent duplicate accounts, and stores user information. The system also coordinates with the Permission Manager to store the user's initial privacy and sensor preferences (GPS, accelerometer, and automated detection). Upon successful registration, a confirmation email is sent by the Notification Manager to the user, and the account is activated only after the user verifies the email.
- **Authentication Handler**: This component manages user login and logout operations. It verifies email and password combinations against stored credentials in the database, generates secure session tokens.
- **Password Recovery Handler**: This component manages password reset functionality. It coordinates with the Notification Manager to send password recovery emails with reset links, to allow password changes, and updates user passwords in the database.

2.2.5. Profile Manager

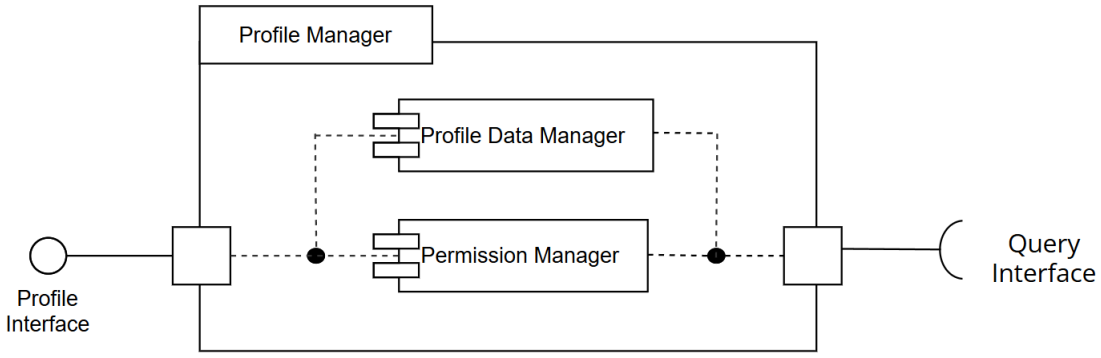


Figure 5 - Profile Manager component

The Profile Manager is composed of two specialized sub-components:

- **Profile Data Manager**: This component is responsible for managing user profile information. It allows registered users to view their current personal information such as email and password, edit profile details, and stores the updated information in the database.
- **Permission Manager**: This component manages device permission settings. It allows users to configure push notification preferences, GPS access, accelerometer and gyroscope sensor access, as well as enable the automated detection mode. All settings are stored in the database.

2.2.6. Searching Manager

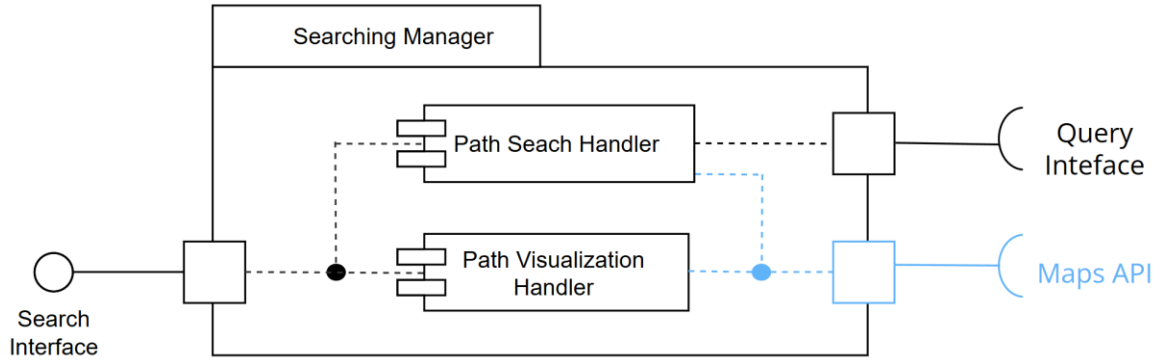


Figure 6 - Searching Manager component

The Searching Manager is composed of two specialized sub-components:

- **Path Search Handler**: This component processes bike path search queries from users. It accepts origin and destination addresses, uses geocoding services from the Mapping Server to convert them into coordinates, queries the database for published bike paths that connect the specified locations, ranks available paths by their score in descending order to present the best options first, and handles cases where no paths are found by informing users appropriately.

- **Path Visualization Handler**: This component displays search results on an interactive map when the user selects the "View on Map" option. It renders bike paths with route overlays, showing multiple alternative routes simultaneously.

2.2.7. Manual Insertion Manager

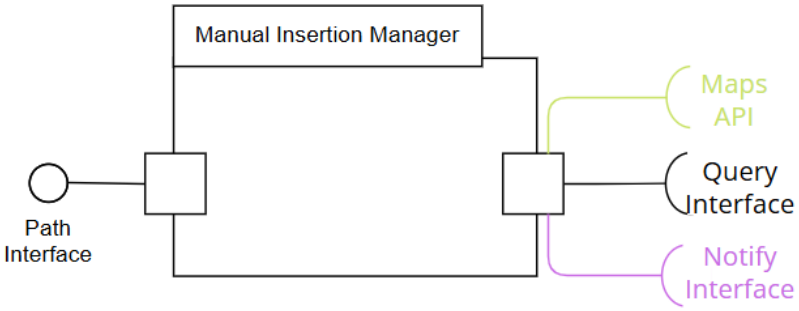


Figure 7 – Manual Insertion Manger component

The Manual Insertion Manager handles manually entered path information from registered users. It allows entering street sequences, validates addresses via mapping services, lets users assign path status and report obstacles, and stores the data.

2.2.8. Collector Manager

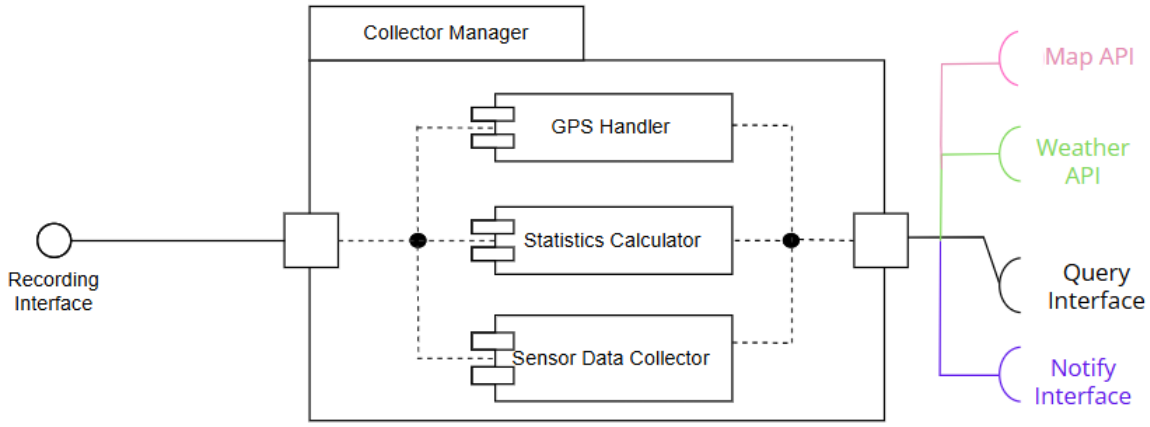


Figure 8 - Collector Manager component

The Collector Manager is used to handle both trip recording and automatic detection mode. It is composed of three specialized sub-components:

- **GPS Handler:** This component manages GPS tracking during recording. It continuously monitors GPS coordinates from the mobile device when the user initiates recording or starts cycling, builds the route by computing distances between consecutive points, handles signal loss with appropriate fallback

mechanisms. The component adjusts how frequently it records GPS points to maintain a balance between location accuracy and battery consumption. All collected GPS data remains in memory and only persisted to the database when the user completes the review flow and confirms saving the trip.

- **Sensor Data Collector**: This component is responsible for collecting and processing accelerometer and gyroscope data from the device sensors when permissions are granted. It detects potential obstacles, correlates sensor events with GPS coordinates, and temporarily stores detected data in memory until the user confirms saving.
- **Statistics Calculator**: This component is responsible for computing trip statistics and performance metrics in real-time from data provided by the GPS Handler and Sensor Data Collector. It calculates total distance travelled based on GPS coordinates, computes trip duration and average speed, and computes derived metrics such as the overall status of the route. It prepares the complete trip summary for user review and enriches it with weather information when available before any data persistence.

2.2.9. Notification Manager

This component manages all user notifications through two distinct channels. For authentication-related communications, it generates and sends email notifications via the Mail Server for account verification and password recovery. For real-time application alerts, it sends push notifications through the app.

2.2.10. Weather Integration Manager

This component manages integration with the Weather Server. It retrieves meteorological data (weather conditions, temperature, wind speed) when internet connectivity is available during trip saving. The component ensures that trips can be recorded and saved even without weather data enrichment.

2.2.11. Database Query Manager

This component serves as the central interface between the BBP Server and the DBMS. It executes all database operations including reading published bike paths, writing new trip and path data, updating user profiles and permissions. The component converts database query results into application objects used by other components.

2.3. Deployment View

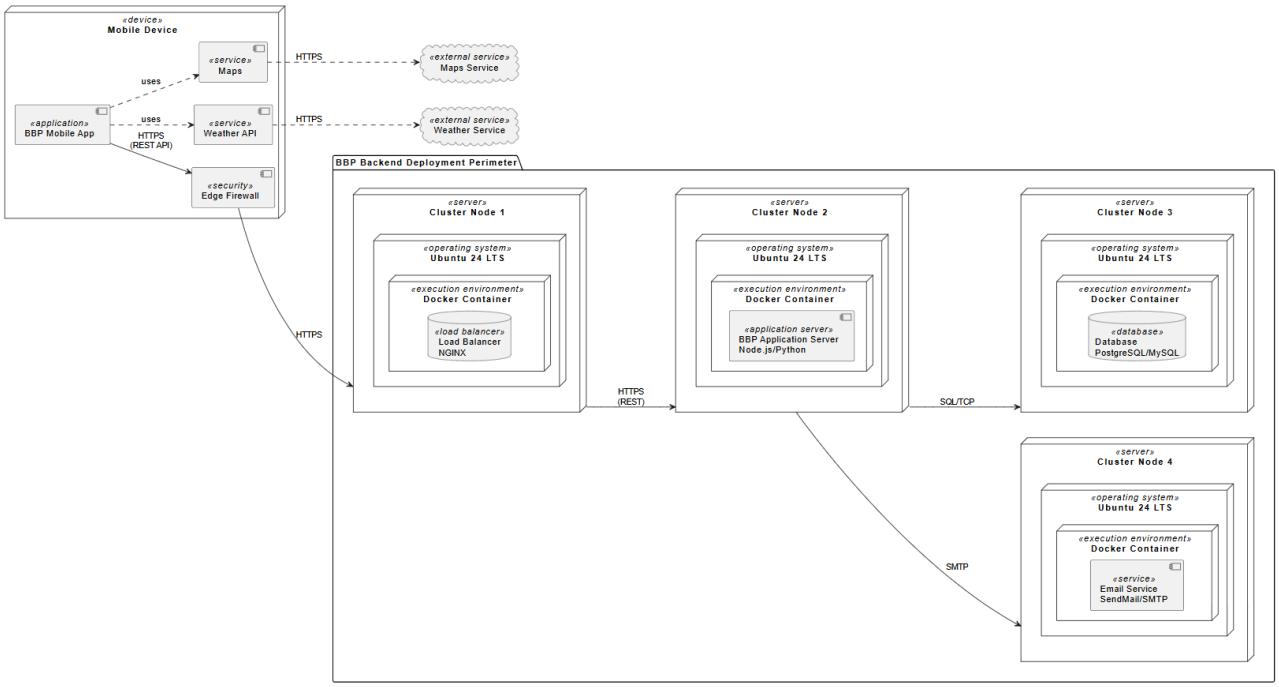


Figure 9 - Deployment diagram

This deployment diagram illustrates the physical architecture of the BBP system, showing how software components are distributed across hardware nodes and execution environments.

Client Side

Mobile Device: Represents the user's smartphone running the BBP mobile application. The device hosts several components:

- **BBP Mobile App**: The main application that users interact with to record trips, browse bike paths, and query routes
- **Maps Service**: Provides mapping and geolocation functionalities
- **Weather API**: Supplies weather information to enrich trip data
- **Edge Firewall**: A security component that protects the device and ensures secure communication with backend services

Backend Infrastructure

The backend is organized within a **BBP Backend Deployment Perimeter**, consisting of multiple cluster nodes running in a distributed architecture:

Cluster Node 1 - Load Balancer:

- Hosts a **NGINX Load Balancer** running in a Docker container on Ubuntu 24 LTS. This component acts as a traffic manager and entry point for all incoming

requests from mobile devices. It distributes incoming HTTPS requests across multiple application server instances to ensure optimal resource utilization, prevent server overload, and provide high availability. If one application server fails, the load balancer automatically redirects traffic to healthy servers, ensuring uninterrupted service.

Cluster Node 2 - Application Server:

- Hosts the **BBP Application Server** (Node.js/Python) in a Docker container. This is the core business logic layer that processes trip recordings, calculates path scores, manages user requests, and coordinates communication with other backend services.

Cluster Node 3 - Database:

- Hosts the **Database** (MySQL) in a Docker container. It stores all persistent data including user profiles, trip records, bike path information, and obstacle reports.

Cluster Node 4 - Email Service:

- Hosts the **Email Service** (SMTP) in a Docker container. It handles all email notifications sent to users, such as trip confirmations, path alerts, and community updates.

External Services

Maps Service: An external cloud service that provides geospatial data, routing algorithms, and map visualization capabilities. The mobile app's Maps component communicates with this service via HTTPS to retrieve map tiles and calculate routes.
Weather Service: An external cloud service that provides meteorological data including weather conditions, temperature, wind speed, and precipitation. The mobile app communicates with this service via HTTPS through the Weather API.

Communication Protocols

- **HTTPS (REST API)**: Secure communication between mobile devices and backend services
- **SMTP**: Email protocol for sending notifications
- **SQL/TCP**: Database connection protocol for data persistence operations

2.4. Runtime View

This chapter describes the behaviour and interactions of the BBP system's building blocks as runtime elements (processes, threads, etc.). It illustrates how the system executes specific scenarios and use cases, showing the dynamic aspects of the architecture.

1. **Login**

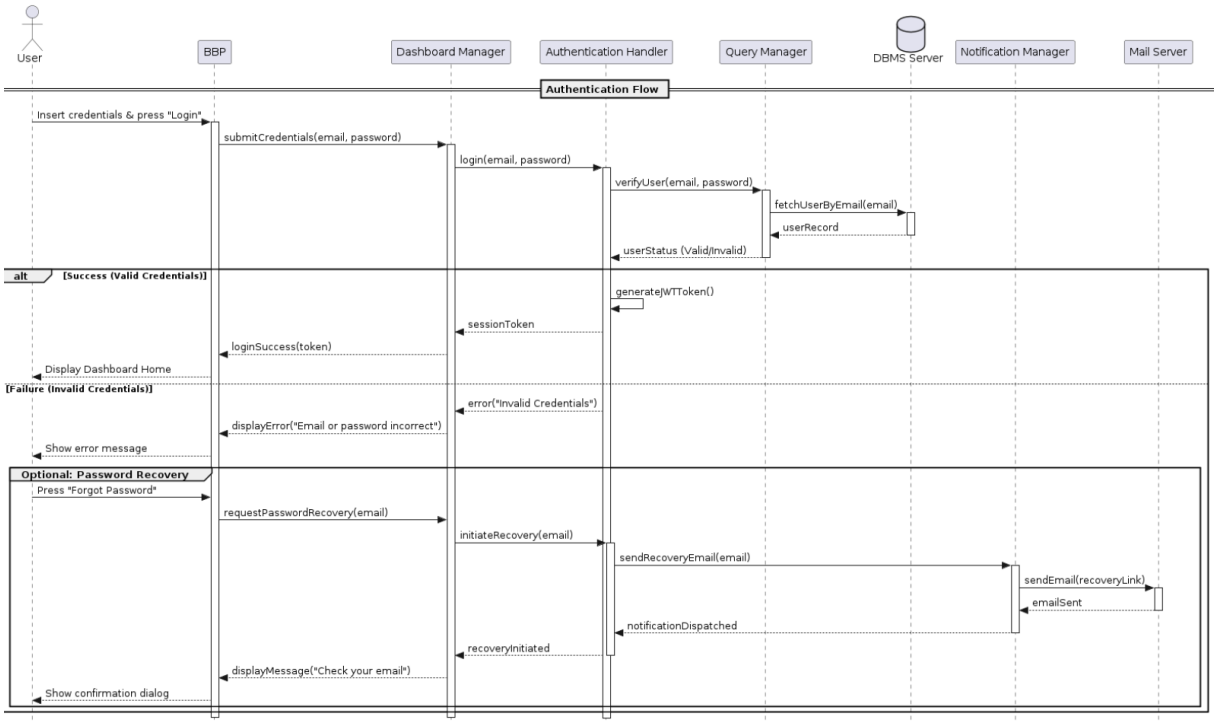


Figure 10 - Login sequence diagram

This sequence diagram illustrates the workflow of the authentication process, highlighting the interaction between the system's core components.

When the user provides their credentials, the Dashboard Manager orchestrates the request by invoking the Authentication Handler, which utilizes the Query Manager to securely verify the user's data against the DBMS Server.

The process ensures that session management, through the generation of a JWT (JSON Web Token), is handled only upon successful verification. Furthermore, the diagram depicts the integration with the Notification Manager and the external Mail Server to manage password recovery requests.

2. Registration

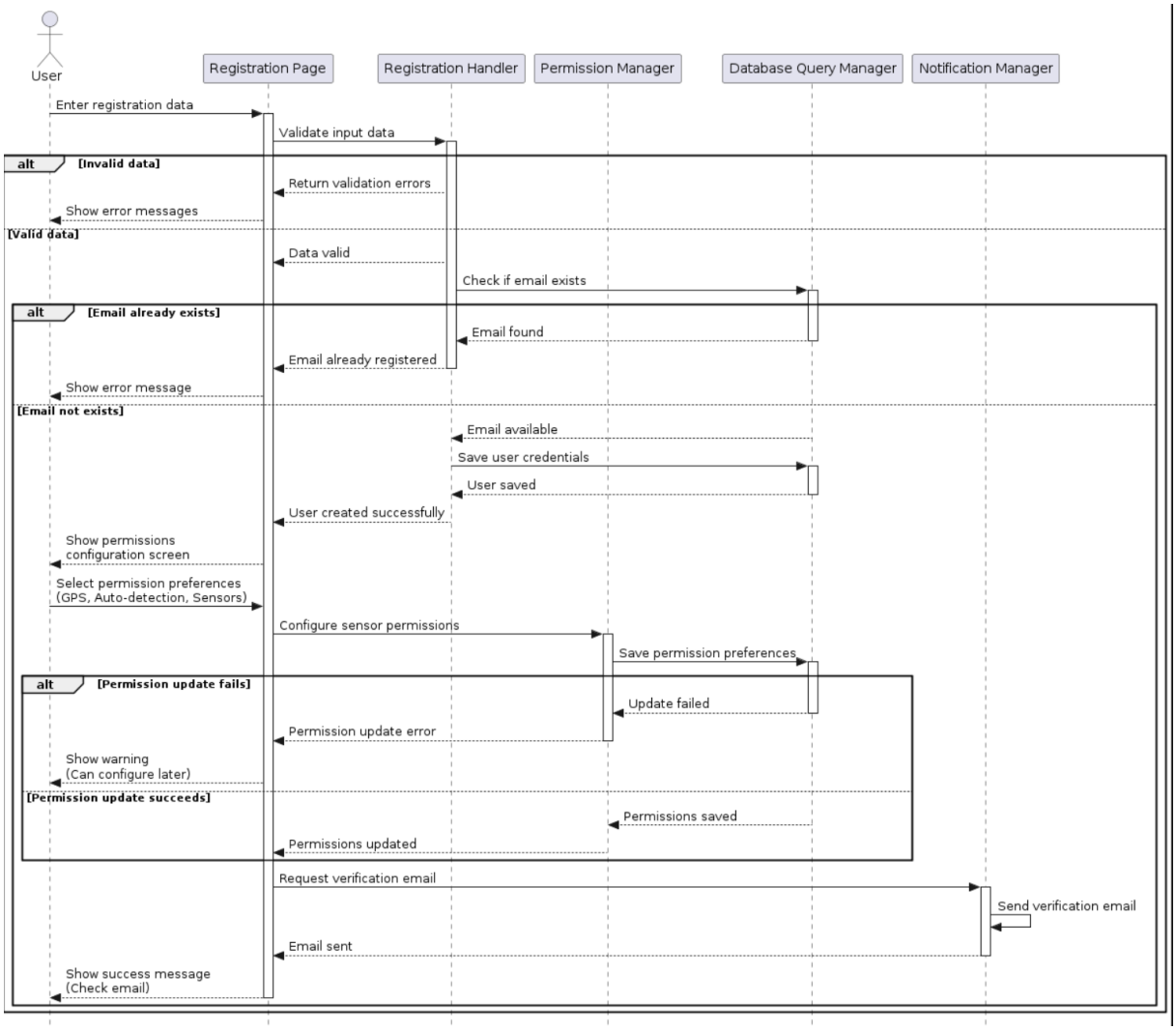


Figure 11 - Registration sequence diagram

This sequence diagram shows the registration workflow and interactions between system components. The user enters registration data on the Registration Page, which sends the data to the Registration Handler for validation. If the data is invalid, error messages are returned to the user.

If the data is valid, the system checks through the Database Query Manager whether the email already exists. If it does, the user is notified of the error. Otherwise, the user credentials are saved and the account is created.

After successful registration, the user configures permission preferences. These are processed by the Permission Manager and stored in the database. If saving permissions fails, the user receives a warning; otherwise, the permissions are confirmed.

Finally, the Notification Manager sends a verification email, and the user is informed to check their email to complete the registration.

3. Start/End recording

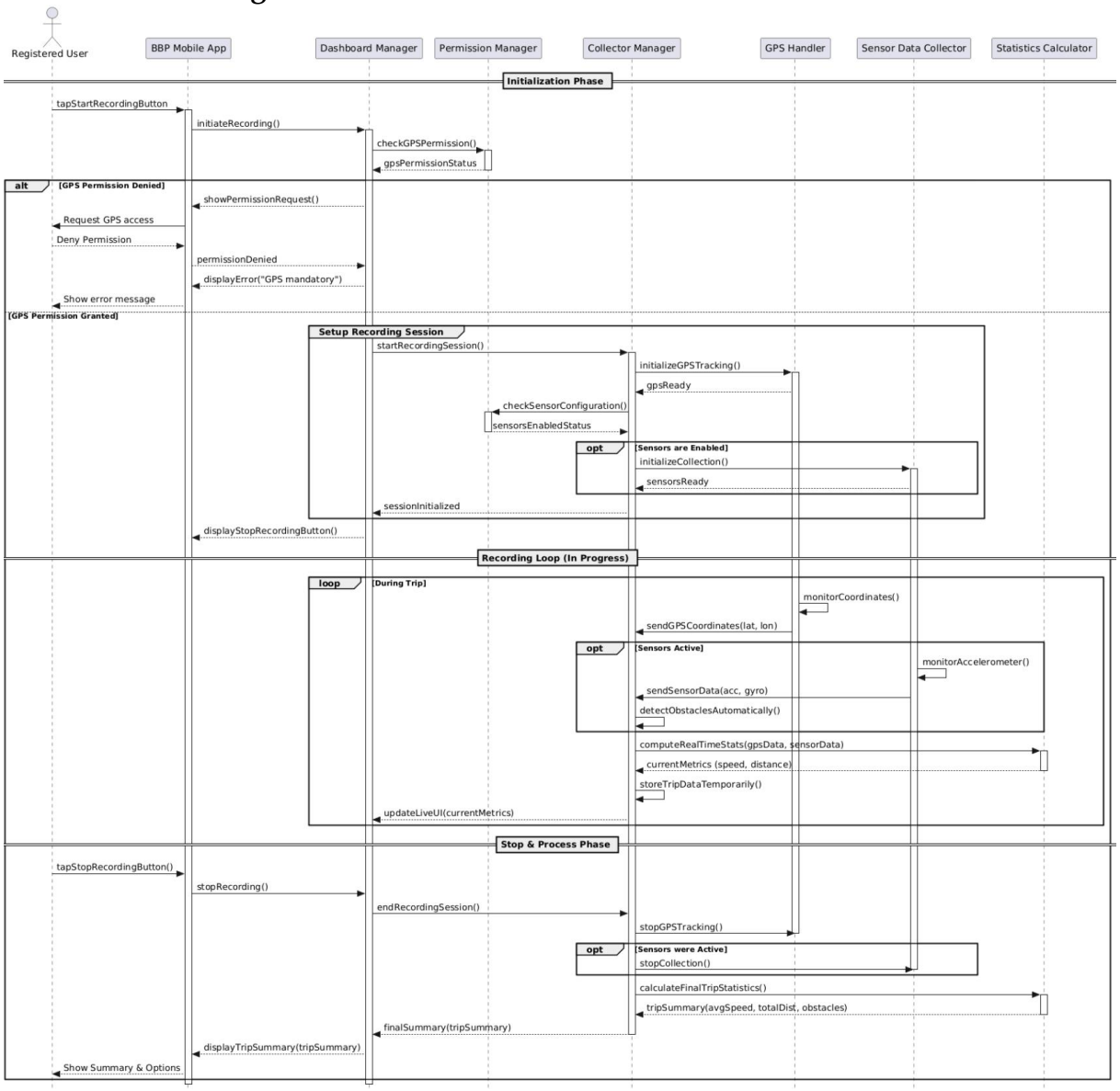


Figure 12 - Manual trip recording sequence diagram

This sequence diagram describes the trip recording lifecycle and the orchestration between the Dashboard Manager and the Collector Manager. The process begins with a mandatory check where the Dashboard Manager consults the Permission Manager to verify GPS access. Once confirmed, the Collector Manager takes control of the session by initializing both the GPS Handler and the Sensor Data Collector according to the user's settings.

During the recording loop, the GPS Handler and Sensor Data Collector continuously feed raw data to the Collector Manager, which delegates real-time performance metrics to the Statistics Calculator. The process concludes with a data aggregation phase where the Statistics Calculator produces a final summary for user review, allowing the user to either discard the temporary records or proceed to the editing and saving flow.

4. Search and visualize on map

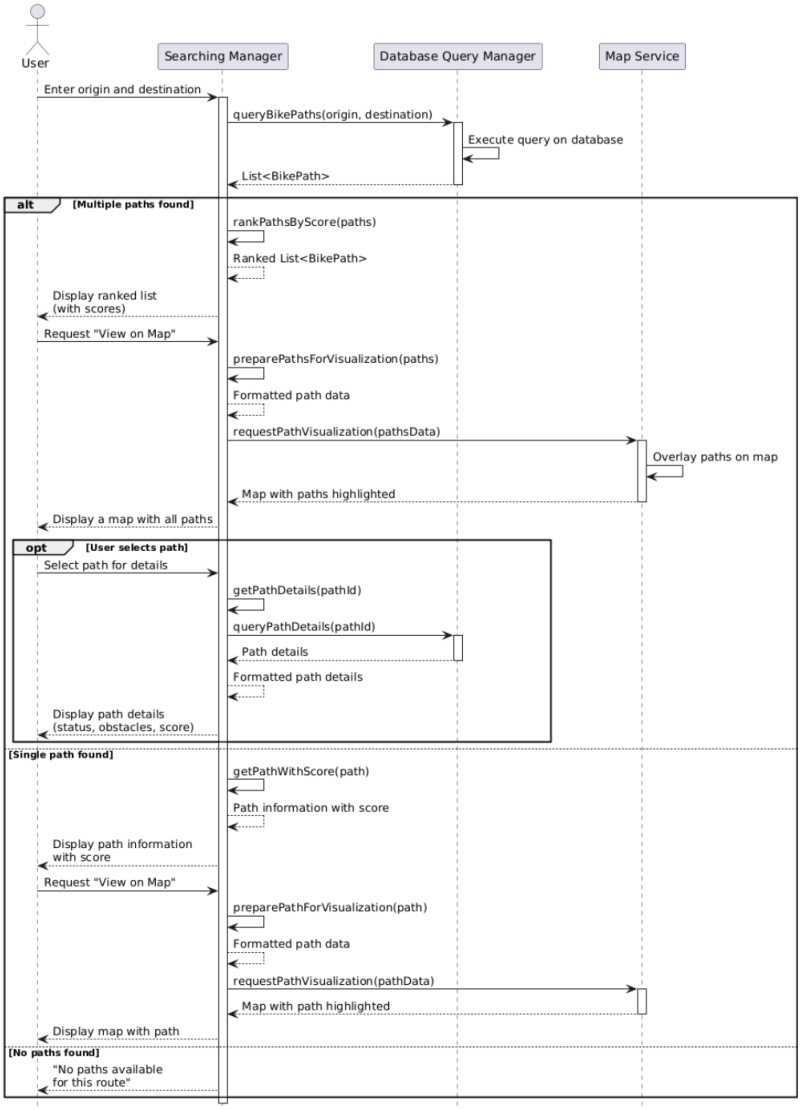


Figure 13 - Searching sequence diagram

This sequence diagram describes how a user searches for bike paths and visualizes them on a map. The process starts when the User enters an origin and destination into the Searching Manager, which queries the Database Query Manager to retrieve available bike paths.

If multiple paths are found, the Searching Manager ranks them on their calculated score and displays the ranked list to the user. When the user requests to view the paths on the map, the paths are prepared for visualization and sent to the Map Service, which overlays them on the map and returns the visual result. The user can optionally select a specific path to view detailed information such as status, obstacles, and score.

If only one path is found, its information and score are retrieved and shown directly to the user. The user can then request to view the path on the map, which is visualized through the Map Service.

If no paths are available for the selected route, the system informs the user that no bike paths are found.

5. Visualize My Trips

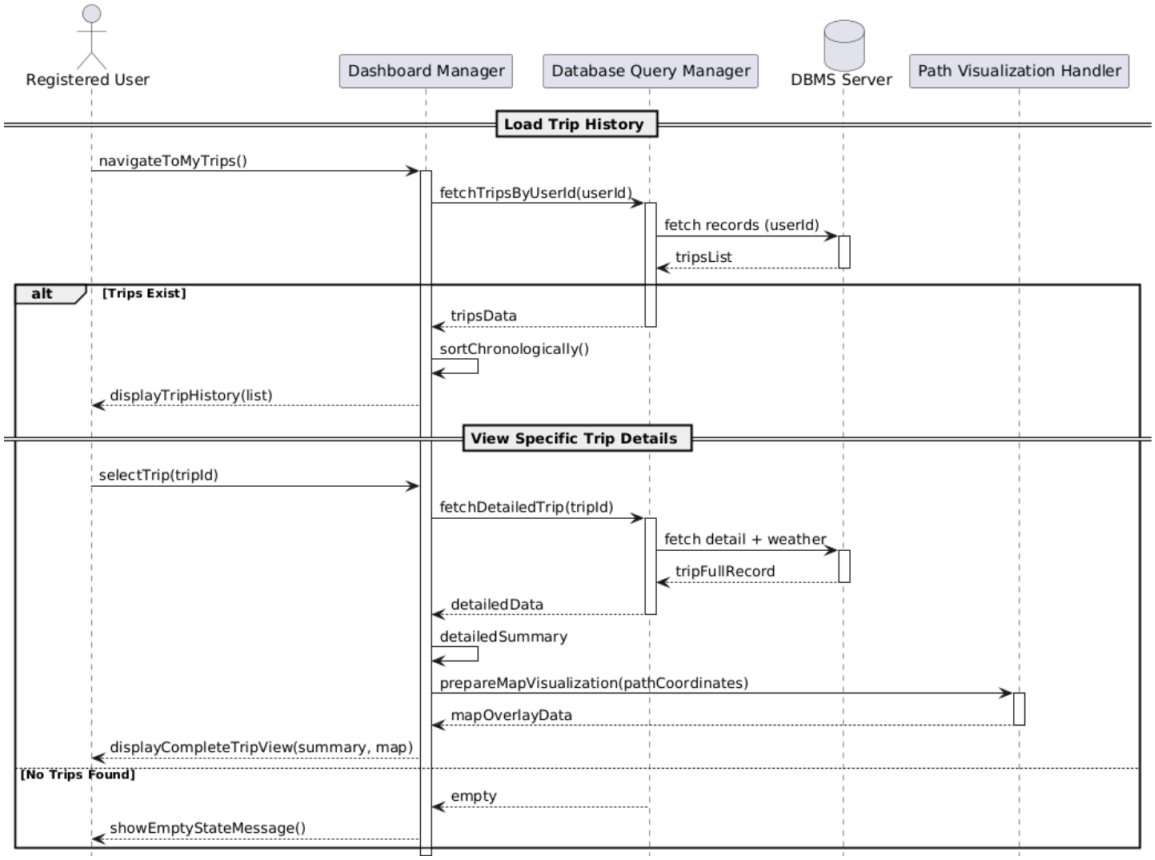


Figure 14 - My trips sequence diagram

This runtime view explains how the system handles the retrieval and display of historical trip data. The process is managed by the Dashboard Manager, which acts as the direct interface for user actions within the application.

When the user requests their trip history, the Dashboard Manager coordinates with the Database Query Manager to extract all relevant records from the DBMS Server. The Dashboard Manager sort these records chronologically to ensure a clear presentation. Once a specific trip is selected, a secondary detailed query is performed to retrieve GPS coordinates and weather information. The Path Visualization Handler is then invoked to transform the raw coordinates into a visual map overlay.

6. Automated detected path

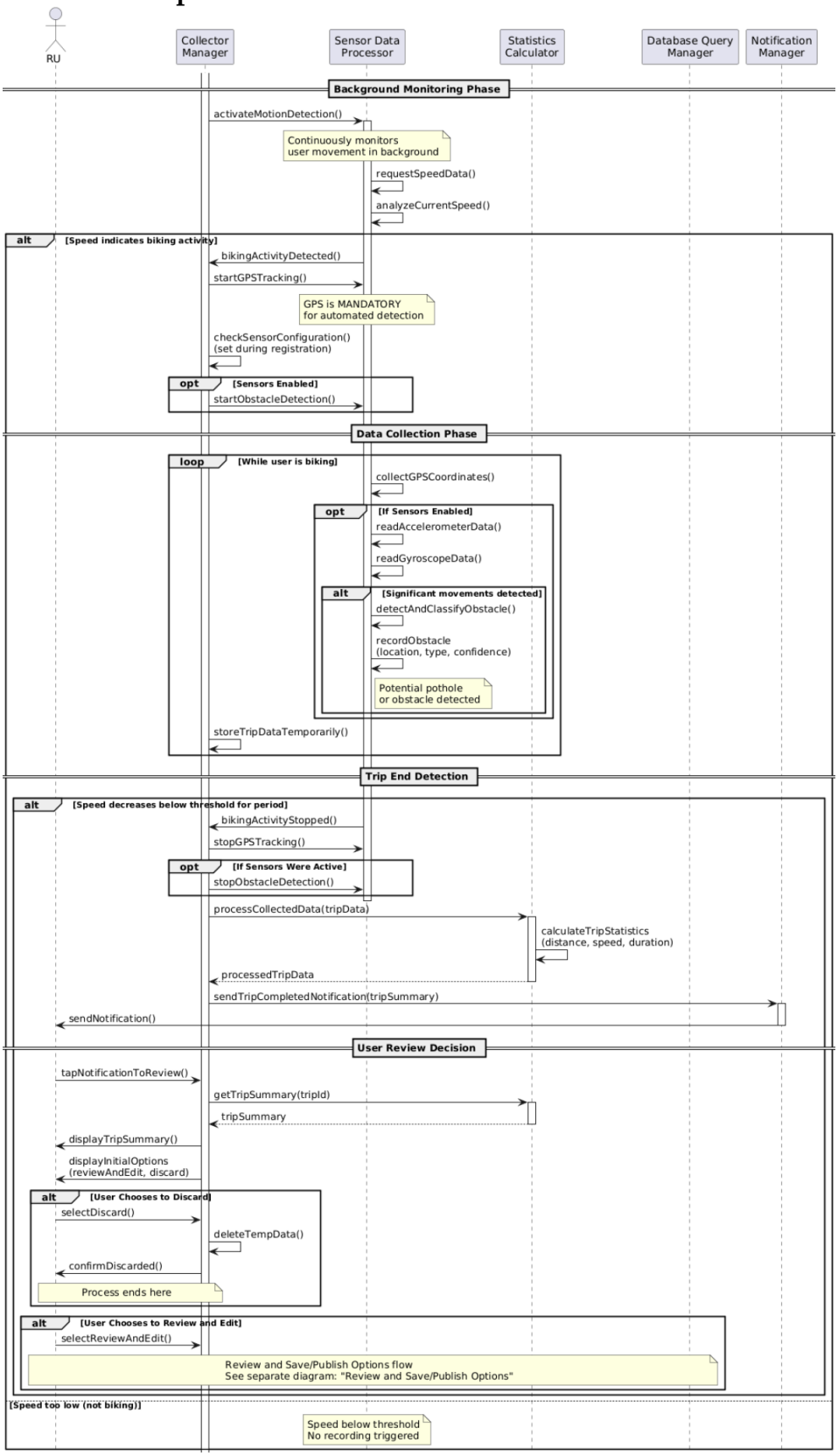


Figure 15 - Automated detection sequence diagram

This sequence diagram illustrates the automated detection of a bike trip using background monitoring and sensor data. The process begins with the Collector Manager activating motion detection through the Sensor Data Collector, which continuously monitors the user's movement.

When the sensor data indicates biking activity, GPS tracking is started, as GPS is mandatory for automated detection. The system checks the sensor configuration defined during registration, and if enabled, obstacle detection is also activated.

During the data collection phase, the system continuously gathers GPS coordinates while the user is biking. If additional sensors are enabled, accelerometer and gyroscope data are analysed to detect and classify potential obstacles, which are recorded with their location and suggested type. Trip data is stored temporarily throughout the ride.

When biking activity stops, GPS and sensor tracking are halted. The collected data is processed by the Statistics Calculator, which calculates trip statistics such as distance, speed, and duration. The Notification Manager then sends a trip completion notification to the user.

The user can tap the notification to review the trip summary. At this point, the user may either discard the trip, which deletes the temporary data, or choose to review and edit it further, leading to a separate review and save/publish workflow.

7. Insert path info manually

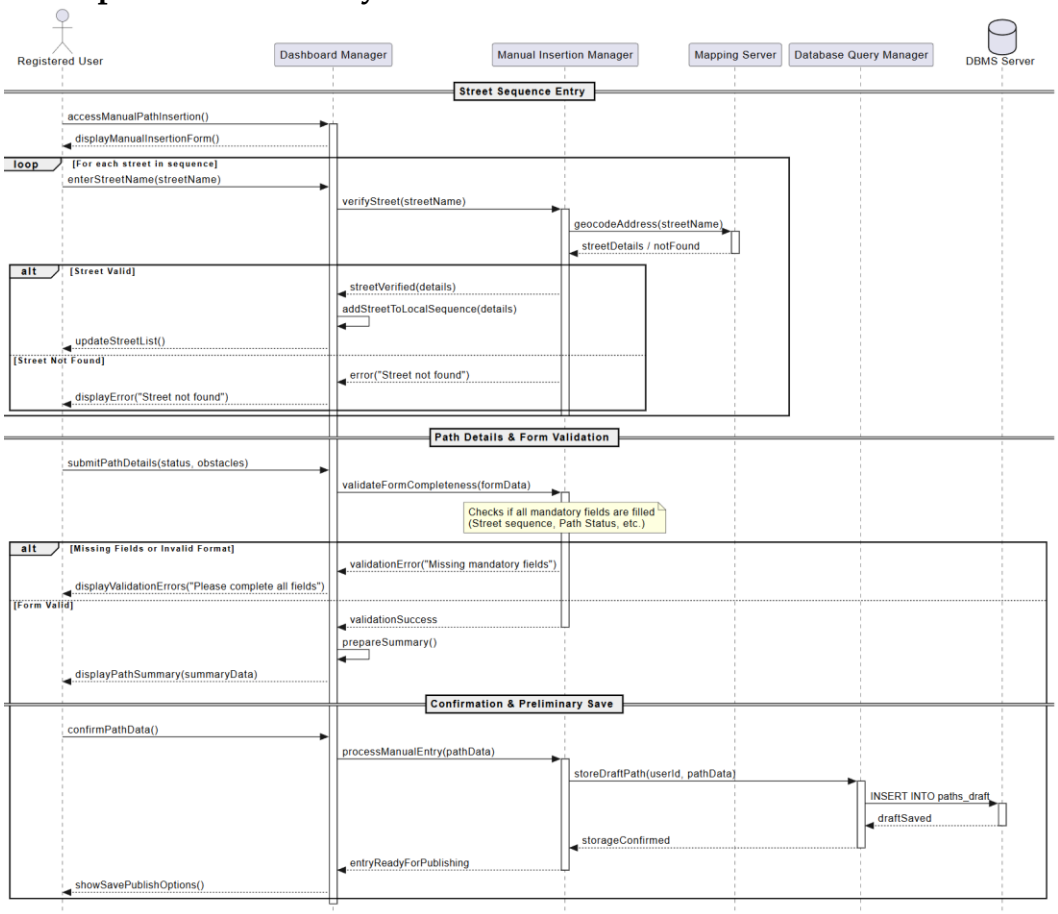


Figure 16 – Manual path insertion sequence diagram

This diagram explains the logic behind the manual entry of bike path information, ensuring both geographical accuracy and data completeness. The Dashboard Manager acts as the primary interface, collecting street names and qualitative path details from the user.

To ensure that the entered streets are valid, the Manual Insertion Manager communicates with the external Mapping Server for each entry. A critical step occurs during the final submission, where the Manual Insertion Manager performs a form completeness check. This ensures that all mandatory information—such as the full street sequence and the overall path status—is correctly provided before any data persistence occurs. Once validated, the Database Query Manager stores the information in the DBMS Server.

8. Review and save

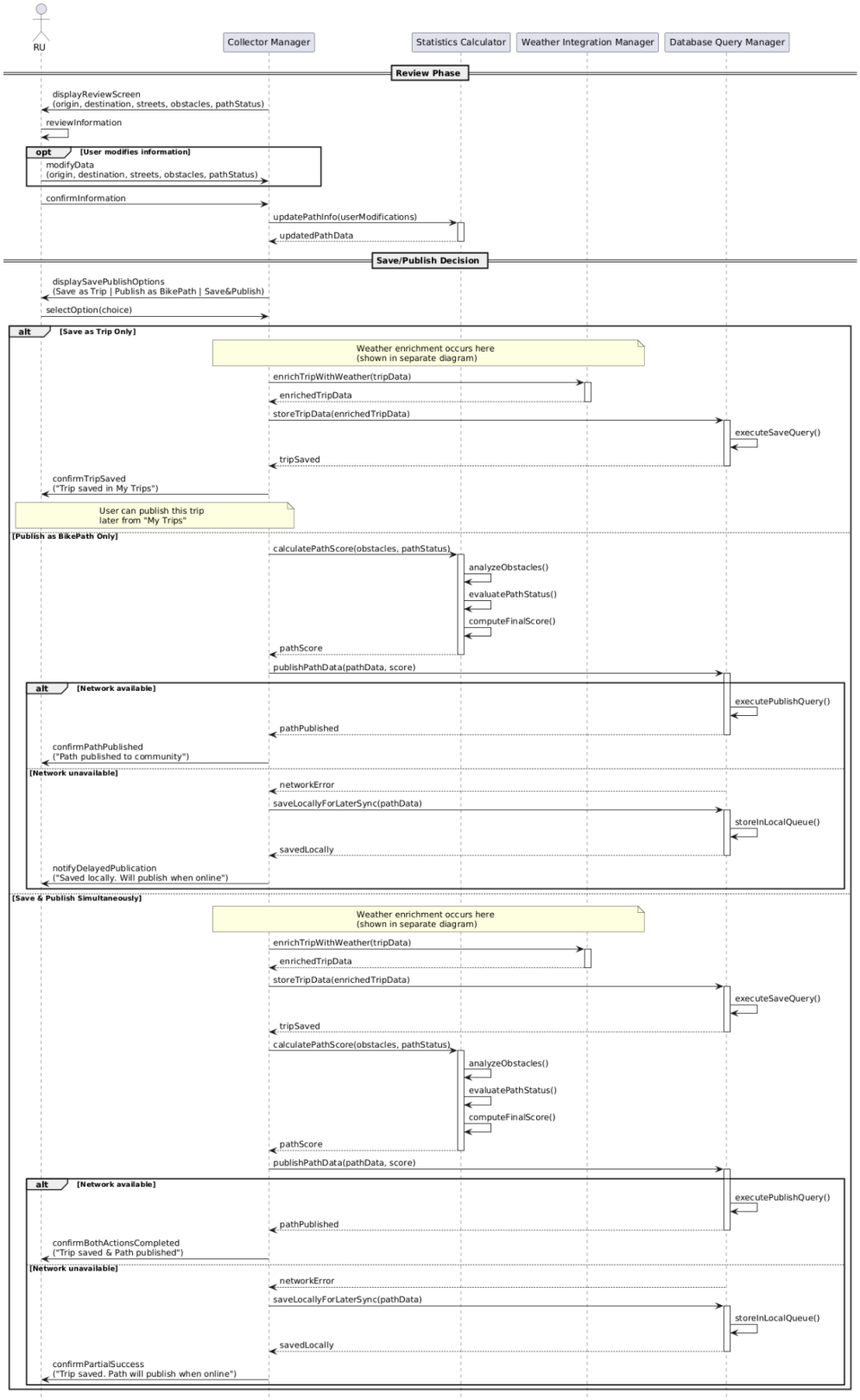


Figure 17 – Review, save and publication options sequence diagram

This sequence diagram describes the workflow that allows a user to review, edit, and decide how to store or publish a recorded bike trip. The process starts when the Collector Manager displays a review screen showing the detected path information, including origin, destination, streets, obstacles, and path status. The user can optionally modify this information and then confirm it.

The confirmed data is sent to the Statistics Calculator, which updates and returns the finalized path information.

Next, the user is presented with three options: save the trip, publish it as a bike path, or save and publish simultaneously.

If the user chooses to save the trip only, the trip data is enriched with weather information through the Weather Integration Manager and then stored in the database. The user is notified that the trip has been successfully saved.

If the user chooses to publish the path only, the Statistics Calculator calculates a path score based on obstacles and path status. The path data is then published to the database. If the network is unavailable, the data is saved locally for later synchronization, and the user is informed of the delayed publication.

If the user chooses to save and publish, the system first enriches and saves the trip data, then calculates the path score and attempts to publish the path. Depending on network availability, the user is either notified of full success or informed that the path will be published later while the trip has already been saved.

9. Weather info enrichment

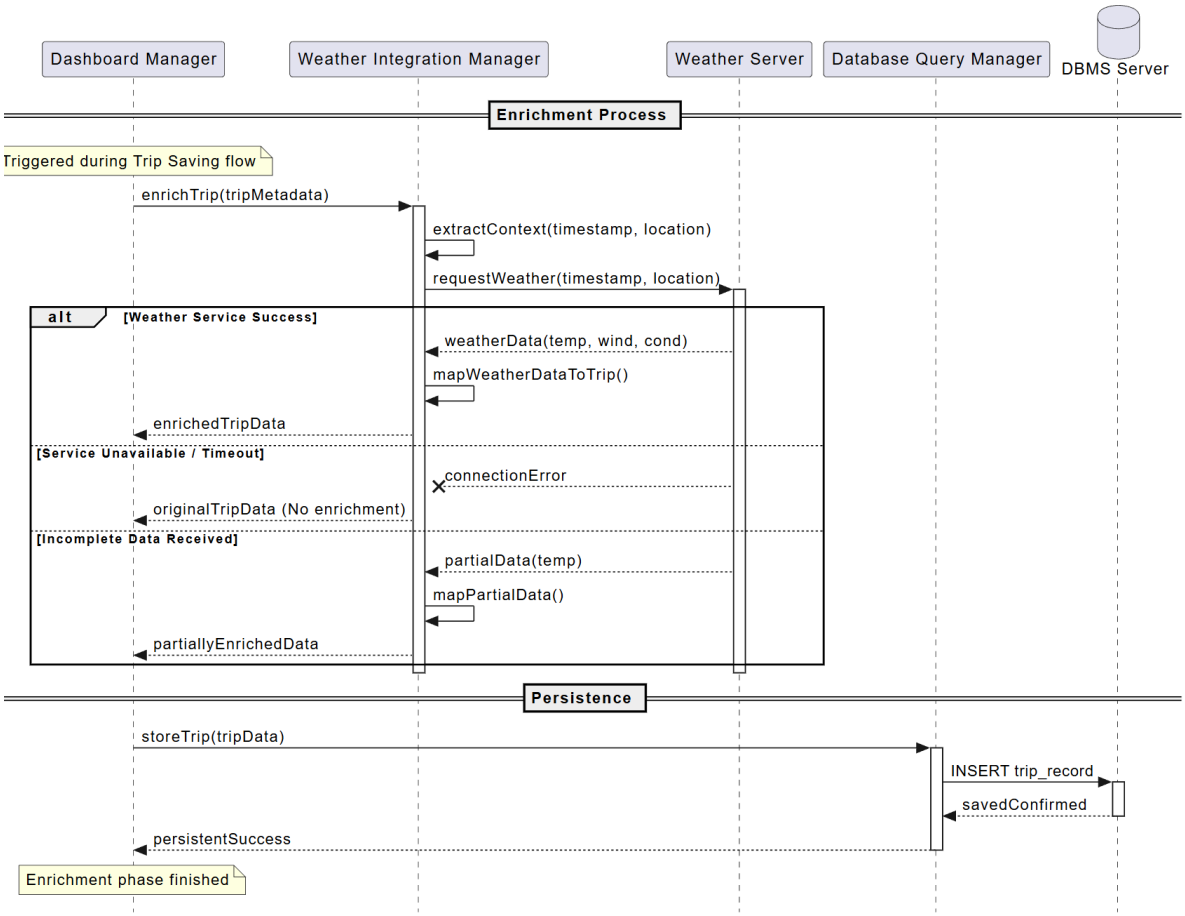


Figure 18 - Weather enrichment sequence diagram

This runtime view illustrates the background process of enriching trip records with meteorological data. The process begins after the trip recording is completed, when the Dashboard Manager passes the trip's metadata (timestamp and GPS coordinates) to the Weather Integration Manager. This component then communicates with the external Weather Server via an API request. The system is designed to handle three distinct scenarios: a full data retrieval, a partial retrieval (e.g., only temperature is available), or a total service failure. In all cases, the Weather Integration Manager ensures that the trip recording is still valid; if the weather data is missing or incomplete, it simply returns the trip record without the additional information. Finally, the Database Query Manager persists the resulting data into the DBMS Server.

2.5. Component Interfaces

This section defines the interfaces exposed by each component of the BBP Application Server. These interfaces specify the operations available to other components and external systems, establishing the contracts for interaction within the architecture.

2.5.1. Authentication Manager

- **registerUser(String email, String password, Boolean GPSEnabled, Boolean SensorEnabled)**
  - Registers a new user account in the system
  - Validates email uniqueness and password strength
  - Ask the user for the permissions needed
- **login(String email, String password)**
  - Authenticates user credentials
  - Generates and returns session token upon successful authentication
  - Returns authentication failure status if credentials are invalid
- **logout(String sessionToken)**
  - Terminates the user session
  - Invalidates the session token
- **requestPasswordReset(String email)**
  - Initiates password recovery process
  - Sends password reset email via Email Service

2.5.2. Collector Manager

- **startTripRecording(String userId, GPSCoordinates startLocation)**
  - Initiates trip recording session
  - Activates GPS tracking and sensor data collection
  - Returns trip ID for the new recording session
- **stopTripRecording(String tripId, GPSCoordinates endLocation)**
  - Terminates active trip recording
  - Processes and stores complete GPS trace
  - Returns trip summary with basic statistics
- **getTripStatistics(String tripId)**
  - Calculates and returns trip metrics (distance, duration, average speed, elevation)
  - Returns empty result if trip data is incomplete
- **processSensorData(String tripId, SensorData sensorReadings)**
  - Receives and processes accelerometer and gyroscope data during trip recording
  - Stores sensor data for obstacle detection analysis
  - Returns processing acknowledgment

- **saveTrip(String tripId, Boolean savePersonally)**
  - Persists trip record to database as personal trip
  - Associates trip with user account
  - Returns save confirmation status
- **enrichTripWithWeather(String tripId)**
  - Retrieves weather data from Weather Service based on trip timestamp and location
  - Enriches trip record with meteorological information (temperature, conditions, wind speed)
- **startPathDetection(String userId): String**
  - Starts a new automated path detection session
  - Activates GPS and sensors to collect data during the trip
  - Returns a unique tripId to identify this session
- **stopPathDetection(String tripId)**
  - Terminates the detection session
  - Processes all collected GPS and sensor data
  - Prepares data for path analysis
- **detectAutomatedPath(String tripId): DetectedPathResult**
  - Analyzes trip GPS trace to identify potential bike paths
  - Applies path detection algorithm to GPS and sensor data
  - Returns detected path segments with confidence scores, suggested origin/destination addresses, and automatically detected obstacles
- **getDetectionStatus(String tripId): DetectionStatus**
  - Retrieves current status of an ongoing detection session
  - Provides information on progress and preliminary data
  - Useful for showing real-time feedback to user
- **confirmDetectedObstacles(String tripId, List<ObstacleData> confirmedObstacles): ConfirmationResult**
  - Receives user confirmation for automatically detected obstacles
  - Validates obstacle data (type, location, severity)
  - Associates confirmed obstacles with path information
  - Returns result with accepted obstacles

2.5.3. Manual Insertion Manager

- **createManualPathEntry(String userId, List<StreetName> streetSequence): PathCreationResult**
  - Receives manual path insertion from user
  - Validates street sequence against mapping database
  - Returns validation status, pathId, and validated coordinates
- **validateStreetSequence(List<StreetName> streets): ValidationResult**
  - Verifies street names against geocoding service
  - Checks if streets form a continuous path

    - Returns list of invalid streets and suggestions if validation fails
- **setPathDetails(String pathId, PathStatus status, SurfaceType surface, DifficultyLevel difficulty, Boolean requiresMaintenance): UpdateResult**
    - Associates metadata and characteristics with path entry
    - Validates status and attribute values
    - Returns update confirmation
- **addObstacleToPath(String pathId, ObstacleType type, GPSCoordinates location, SeverityLevel severity): String**
    - Adds obstacle information to path record
    - Supports multiple obstacles per path
    - Returns unique obstacleId
- **removeObstacleFromPath(String pathId, String obstacleId): Boolean**
    - Removes a specific obstacle from a path
    - Useful for correcting errors or when obstacle has been removed
    - Returns deletion confirmation

PATH MANAGEMENT OPERATIONS

- **getPathDetails(String pathId): PathInfo**
    - Retrieves complete information about a specific path
    - Includes all details: streets, status, obstacles, score, etc.
    - Throws PathNotFoundException if path doesn't exist
- **updatePathStatus(String pathId, PathStatus newStatus): UpdateResult**
    - Updates the status of an existing path
    - Allows reporting changes in path conditions
    - Returns update confirmation
- **getPathsByUser(String userId): List<PathInfo>**
    - Retrieves all paths created by a specific user
    - Useful for displaying user's contribution history
    - Returns list of PathInfo
- **publishPath(String pathId): PublicationResult**
    - Publishes path information to community database
    - Makes path searchable and visible to all users
    - Computes initial path score based on status and effectiveness
    - Returns result with timestamp, initial score, and visibility level
    - Throws PathNotReadyException if required information is missing
- **discardPath(String pathId): Boolean**
    - Deletes unpublished path information
    - Removes associated obstacle data
    - Returns deletion confirmation
    - Throws PathAlreadyPublishedException if path is already published

2.5.4. Searching Manager

- **searchPaths(GPSCoordinates origin, GPSCoordinates destination, SearchFilters filters)**
  - Searches for bike paths between two locations
  - Returns list of matching paths ordered by score
- **getPathDetails(String pathId)**
  - Retrieves complete information for a specific path
  - Returns path metadata, street sequence, obstacles, and score
  - Returns error if path not found
- **displayPathsOnMap(List<String> pathIds)**
  - Prepares path data for map visualization
  - Converts street sequences to GPS coordinates via geocoding
  - Returns map-ready path geometries

2.5.5. Sensor Data Collector

- **analyzeAccelerometerData(String tripId, AccelerometerReadings readings)**
  - Processes accelerometer data to detect bumps and irregularities
  - Identifies potential obstacles based on sudden changes in acceleration
  - Returns list of detected anomalies with timestamps and locations
- **analyzeGyroscopeData(String tripId, GyroscopeReadings readings)**
  - Processes gyroscope data to detect sharp turns or path characteristics
  - Identifies orientation changes that may indicate path features
  - Returns orientation analysis results
- **detectObstacles(String tripId)**
  - Combines accelerometer and gyroscope analysis to identify obstacles
  - Classifies detected obstacles by type (bump, pothole, debris)
  - Returns list of detected obstacles requiring user confirmation

2.5.6. Weather Integration Manager

- **fetchWeatherData(GPSCoordinates location, Timestamp time)**
  - Communicates with external Weather APIs to retrieve meteorological conditions.
- **enrichTripRecord(String tripId, WeatherData weatherInfo)**
  - Updates a specific trip record with the retrieved weather data.

2.5.7. Database Query Manager

- **storeUserProfile(String userId, UserProfile profile)**
  - Persists user account information to database
  - Returns storage confirmation
- **retrieveUserProfile(String userId)**

    - Fetches user profile from database
    - Returns UserProfile object or null if not found
- **storeTripRecord(TripData trip)**
    - Saves trip information (GPS trace, statistics, weather) to database
    - Returns generated trip ID
- **retrieveTripHistory(String userId, Integer limit)**
    - Fetches user's trip history from database
    - Returns list of trip records ordered by date (most recent first)
- **storePathInformation(PathInfo path)**
    - Persists published path data to community database
    - Includes street sequence, obstacles, metadata, and initial score
    - Returns generated path ID
- **queryPathsByLocation(GPSCoordinates origin, GPSCoordinates destination)**
    - Executes spatial database query to find paths between two points
    - Returns list of matching path IDs
- **updatePathScore(String pathId, Float newScore)**
    - Updates path effectiveness score in database
    - Returns update confirmation

2.5.8. Notification Manager

- **sendEmailNotification(String userEmail, String subject, String body)**
    - Sends email notification via Email Service (SMTP)
    - Used for account verification, password recovery, and important updates
    - Returns delivery status
- **sendPushNotification(String userId, String message)**
    - Sends push notification to user's mobile device
    - Used for real-time alerts and confirmations
    - Returns delivery status

2.5.9. External Service Interfaces

The BBP Application Server also interacts with external services through their respective interfaces:

Weather Service Interface:

- Platform-specific native APIs
- Input: GPS coordinates, timestamp
- Output: WeatherData (temperature, conditions, wind speed)

Mapping Service Interface:

- Map Service
- **geocode(String streetName)**: Converts street name to GPS coordinates
- **reverseGeocode(GPSCoordinates location)**: Converts GPS coordinates to street address
- **validateAddress(String address)**: Verifies street address exists in mapping database

2.6. Selected architectural styles and patterns

2.6.1. Architectural Design

The architectural design of BBP is based on the industry-standard **3-Tier Architecture** pattern:

1. Presentation Tier (Mobile Client Layer): This is the upper layer that directly interacts with users through native mobile applications. It focuses on displaying information and capturing user inputs through touch-based interfaces. It handles all client-side interactions including map visualization, trip recording controls, and path browsing interfaces.

2. Application Tier (Business Logic Layer): This layer contains the core business logic and processing rules of BBP. It receives requests via REST API, applies business rules (such as path validation, trip statistics calculation, and obstacle detection algorithms), and coordinates the flow of information.

3. Data Tier (Persistence Layer): The bottom layer of the architecture, responsible for storing, retrieving, and managing data. It ensures data integrity through ACID transactions and provides efficient data access methods. This layer persists all critical data including user profiles, trip records with GPS traces, published path information, and obstacle reports.

The division of the system into these three layers allows a reduction in complexity, improves maintainability, and facilitates scalability: each layer can be developed, tested, deployed and maintained independently, enabling a more modular and flexible system architecture while also paralleling development efforts across different teams and ensures platform independence, allowing the same application and data tiers to support both iOS and Android clients seamlessly.

The choice of a 3-Tier Architecture is also motivated by the fact that it maps naturally to the **Model-View-Controller (MVC)** pattern, a widely-used design pattern that separates the application into three interconnected components:

- **Model**: Represented by the Data Tier and domain objects (User, Trip, PathInfo, Obstacle)
- **View**: Represented by the Presentation Tier (mobile UI screens and components)
- **Controller**: Represented by the Application Tier manager components (Authentication Manager, Collector Manager, Manual Insertion Manager, etc.)

This pattern is particularly useful for BBP since it allows for a clear separation of concerns and a more modular and maintainable codebase, which is critical given the complexity of managing GPS data, sensor inputs, and real-time trip recording.

2.6.2. Client-Server Communication

BBP is based on a **Client-Server architecture** with the following communication patterns:

- **Mobile to Backend**: Uses REST API over HTTPS. Mobile apps send JSON-formatted requests and receive JSON responses, ensuring security for sensitive data like GPS traces.
- **Backend to Database**: The application tier communicates with the data tier using SQL queries over TCP/IP, providing structured access to relational data.
- **Backend to External Services**:
    - **Email Service**: Communicates via SMTP for sending account verification and password recovery emails
    - **Weather APIs**: Native platform APIs accessed directly by mobile clients (iOS Weather Framework, Android Weather Service)
    - **Mapping Services**: Native SDKs accessed directly by mobile clients (MapKit for iOS, Google Maps SDK for Android)

2.6.3. Service-Oriented Design Pattern

BBP adopts a Service-Oriented approach where major functionalities are encapsulated in distinct manager components. Each manager exposes a well-defined interface, operates independently with minimal coupling, and provides reusable services that can be invoked by different workflows.

2.6.4. Repository Pattern (Data Access)

The Database Query Manager implements the Repository Pattern, providing an abstraction layer between the business logic and data storage. This centralizes data access logic and allows the underlying database technology to be modified without affecting the core business rules.

2.6.5. Offline-First Pattern

BBP implements an **Offline-First** pattern for core functionalities:

- GPS tracking and sensor data collection work without internet connectivity
- Trip data is cached locally on the mobile device during recording
- Synchronization to the backend occurs when connectivity is restored
- This ensures users can record trips even in areas with poor network coverage

This pattern is essential for a cycling application where users may ride through areas with limited or no cellular coverage.

2.7. Other design decisions

2.7.1. Auxiliary Components

As discussed in Section 2.6, the application tier of BBP depends on several auxiliary services that provide additional functionalities beyond the core Application Server. These services are separated for architectural and practical reasons:

Email Service:

- Communicates with the core Application Server using SMTP over TCP/IP
- The Application Server prepares email content and tasks the Email Service to deliver it to recipients
- Used for account verification and password recovery

Weather Service:

- BBP integrates with platform-native weather APIs
- Weather data is fetched based on GPS coordinates and timestamps to enrich trip records
- Communication occurs directly from mobile clients to platform weather APIs

Mapping Services:

- BBP integrates with platform-native map service
- Used for geocoding, address validation with autocomplete, and path visualization
- Communication occurs directly from mobile clients to mapping services

2.7.2. Other integrated aspects

Native Mobile Development

BBP is developed as native mobile applications to ensure efficient access to device sensors and location services. This choice allows optimal performance, seamless integration with platform-specific APIs, and a user experience consistent with mobile operating system guidelines. Native development is particularly suitable for continuous GPS tracking and real-time sensor data acquisition, which are core functionalities of the system.

Backend Deployment and Containerization

The backend of BBP is deployed using container-based technology. Containerization ensures consistency across development and production environments, improves isolation between services, and simplifies deployment and scalability. This approach supports efficient resource usage and enables horizontal scaling when user demand increases.

Client–Server Communication

Communication between the mobile applications and the backend follows a RESTful architectural style over HTTPS. The system exposes resource-oriented APIs that allow clients to manage users, trips, and bike paths. The stateless nature of REST simplifies scalability and supports load balancing across multiple backend instances.

Database Technology

BBP adopts a relational database to manage persistent data. This choice is motivated by the structured nature of the domain data and the need for transactional consistency. Relational databases also provide strong support for geographic data and spatial queries, which are essential for searching and ranking bike paths.

Security Architecture

BBP applies standard security best practices:

- Encrypted communication between clients and server
- Token-based authentication to support stateless and scalable interactions
- Secure storage of user credentials using hashed passwords

These measures protect sensitive user information and ensure secure access to system functionalities.

Offline Support

The system supports offline trip recording on mobile devices. Trip data is temporarily stored locally and synchronized with the backend once network connectivity is restored. This design improves usability in scenarios where internet access is unreliable.

Error Handling and Resilience

BBP is designed to tolerate temporary failures of external services. When optional data (e.g., weather information) is unavailable, the system continues to operate with reduced functionality rather than failing. This approach improves robustness and overall user experience.

Scalability and Load Distribution

Incoming client requests are distributed across multiple backend instances using a load balancing mechanism. Since the system is stateless, requests can be handled by any available instance, enabling horizontal scalability and high availability.

3. User interface design

This chapter presents the user interface design of the BBP mobile application, illustrating the main user flows and corresponding screen mock-ups.

3.1. Welcome and Authentication Flows

3.1.1. Welcome Screen and Access Options

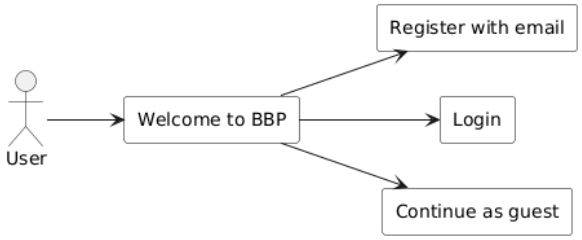


Figure 19 - User access options flow

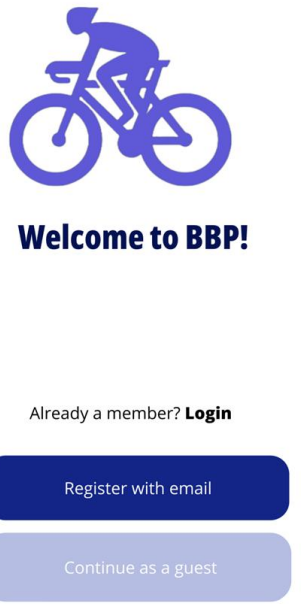


Figure 20 - Welcome screen

The welcome screen displays the BBP logo and brand identity, immediately communicating the application's purpose through cycling-themed visual elements. Users can choose between creating a new account, logging in with existing credentials, or exploring the application as a guest with limited functionality.

3.1.2. Registration Flow

The registration process guides new users through account creation, email verification, and essential permission configuration.

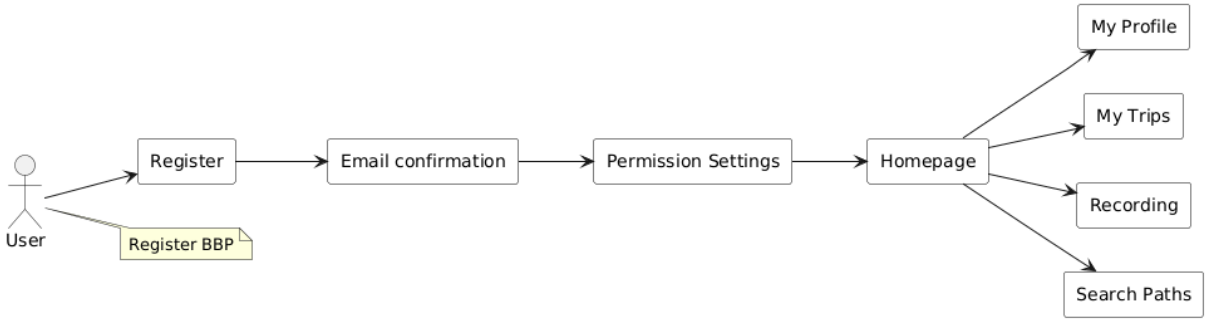


Figure 21- User registration flow

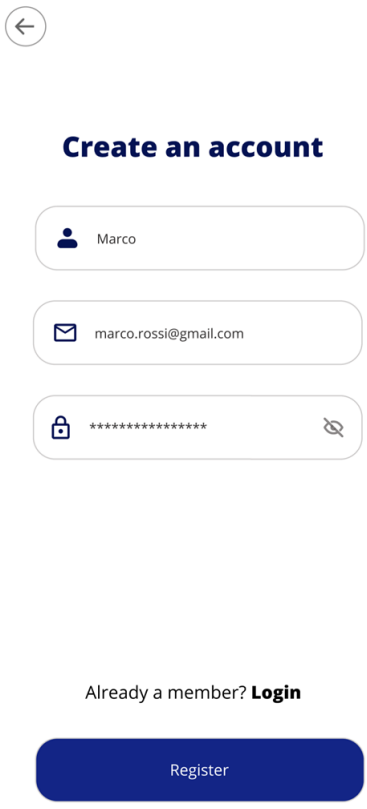


Figure 22 - Account creation screen

Users enter their name email address and choose a secure password. The interface includes password visibility toggle.

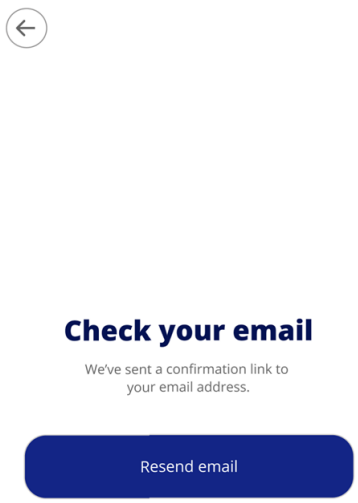
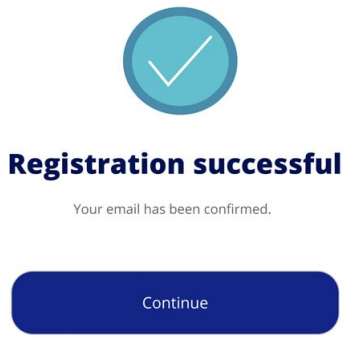


Figure 23 - Email verification prompt

After submitting registration details, users are prompted to check their email inbox for a verification link. The screen includes an option to resend the confirmation email if needed.

 Figure 24 - Registration successful confirmation	Successful registration message.

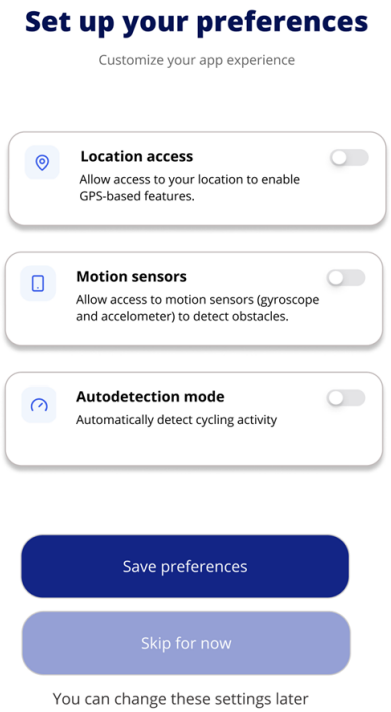


Figure 25 – Permission configuration screen

In the next step the user can configure essential permissions including location access and motion sensors. Users can choose to set up these preferences immediately or skip this step, in both cases they are then redirected to the application homepage.

3.1.3. Login Flow

This flow allows registered users to access their BBP account. After successful authentication, users are redirected to the homepage where they can access all main functionalities.

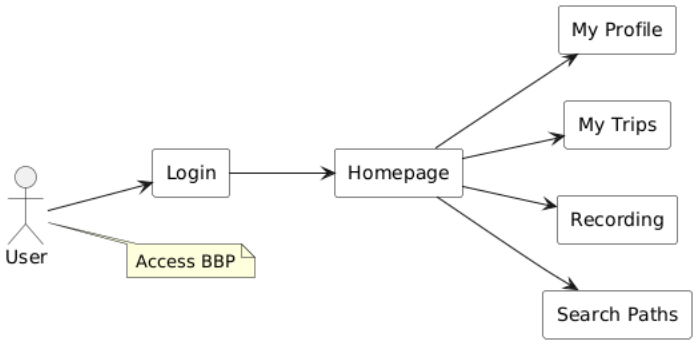


Figure 26 – Registered User login flow

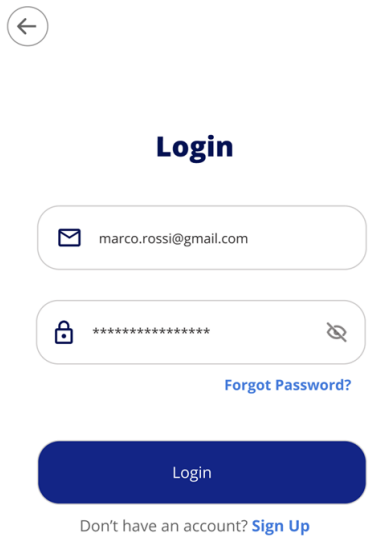


Figure 27 - Login screen

The login screen provides fields for email and password entry, with a password visibility toggle for user convenience. A "Forgot Password?" link enables password recovery for users who cannot recall their credentials. The "Sign Up" link at the bottom allows quick navigation to the registration flow for users who mistakenly selected login instead of registration.

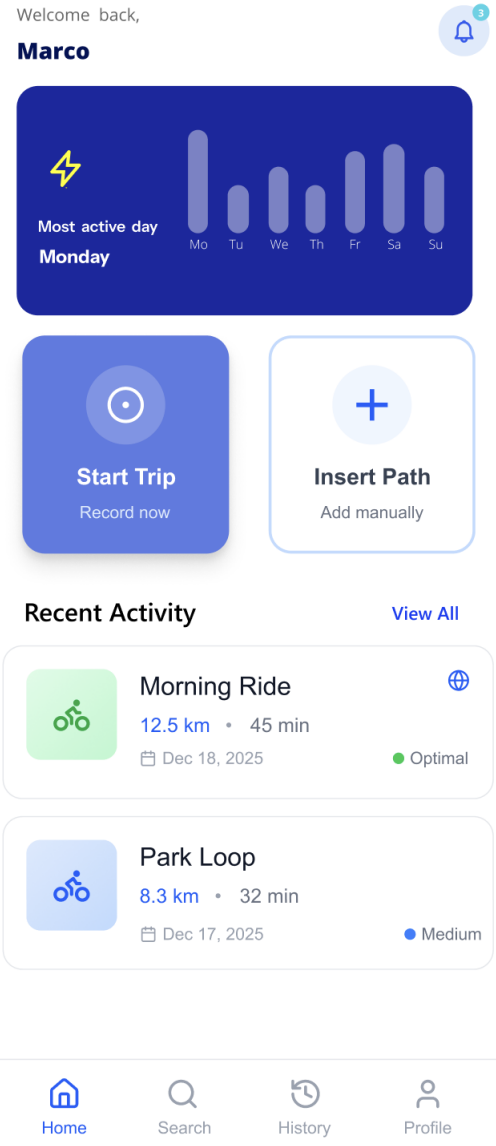


Figure 28 - Registered user homepage

Upon successful authentication, users are immediately redirected to the homepage where they can access all registered user functionalities including trip recording, path insertion, search paths and activity history.

3.2. Guest User Interface

Guest mode allows users to explore BBP's bike path database without creating an account.

3.2.1. Search Bike Paths

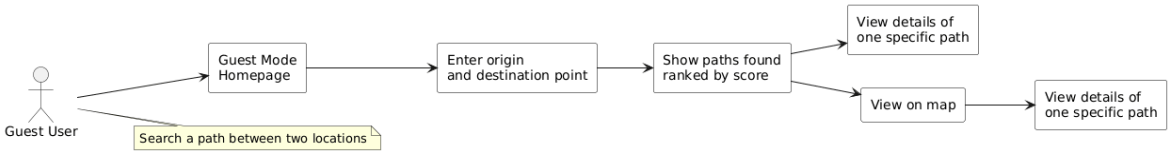


Figure 29- Guest user search flow

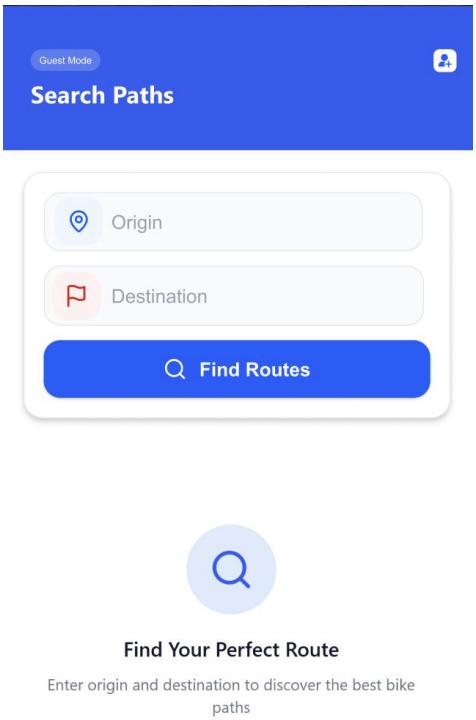


Figure 30 - Guest mode homepage and search interface

This screen serves as the primary homepage for guest users, immediately presenting the core search functionality. A "Guest Mode" badge in the top-left corner clearly indicates the limited access status, while a profile icon in the top-right corner allows users to navigate to the registration screen to unlock full application features.

The search interface provides input fields for origin and destination locations, with the system suggesting locations as users type through integrated map services for address autocomplete. The "Find Routes" button initiates the search once both locations are specified.

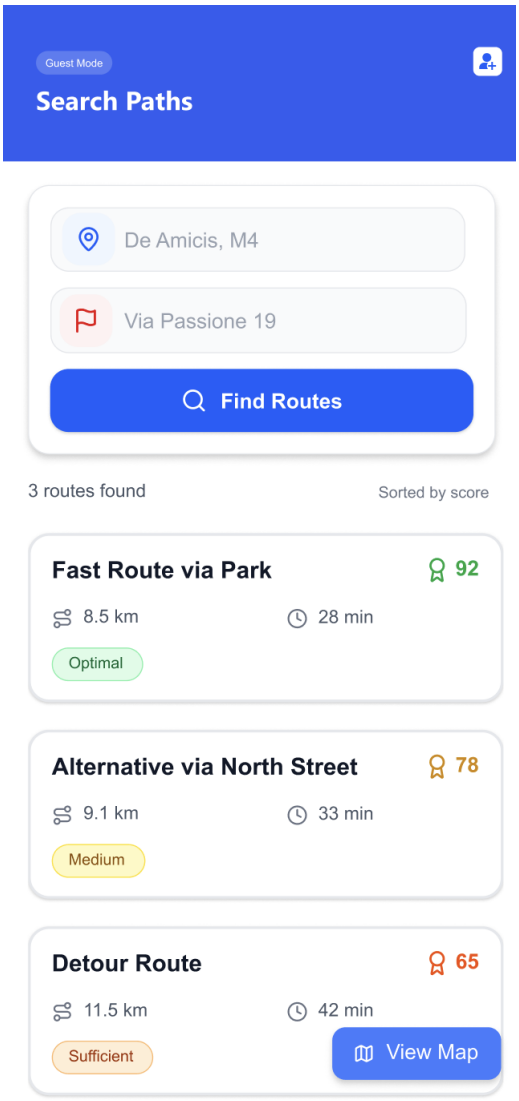


Figure 31 - Ranked search results for guest users

Search results display available paths ranked by quality score. The interface maintains the "Guest Mode" badge and profile icon from the previous screen, allowing users to access registration at any point in their journey.

Each result card shows path name, total distance, estimated duration, quality score and path status. The score and the path status are color-coded: green for Optimal, blue for Medium, yellow for Sufficient, red for Requires Maintenance. Guest users can tap on any result to view detailed information or visualize the paths on the map by clicking the "View Map" button.

3.2.2. Map Visualization

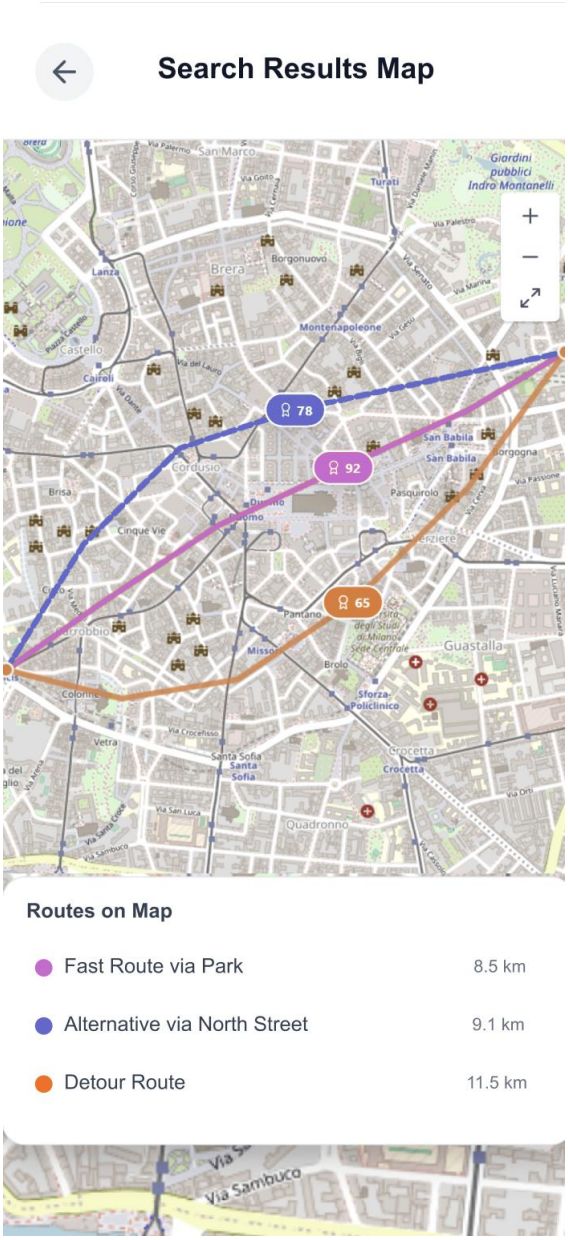


Figure 32 - Map visualization

The map displays available bike paths as coloured overlays, with each route represented by a unique colour and labelled with its quality score. A legend shows which colour corresponds to each route and its total distance, enabling quick path comparison. Users can pan and zoom to explore different areas, and tapping on a path overlay redirects them to the detailed information screen for that specific route.

3.2.3. Path Details

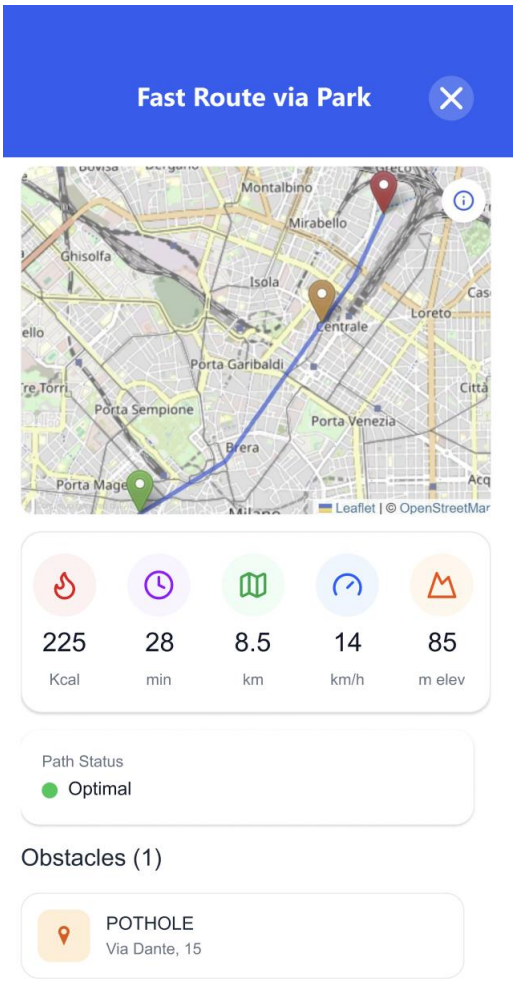


Figure 33 – Path details view

Accessed by tapping a path from search results or map view, this screen provides detailed route information. The header shows the path name with a close button to return to the previous screen. A map preview shows the full route as a blue overlay, with clearly marked points: green marker for the origin point, red marker for the destination point, and orange markers for obstacles along the path.

Below the map, key statistics are shown with intuitive icons: estimated calories burned, duration (minutes), total distance (km), average speed (km/h), and elevation gain (meters). A color-coded path status indicator (e.g., a green dot labelled "Optimal") offers a quick overview of current conditions. The obstacles section lists all reported issues, showing each obstacle type and its location.

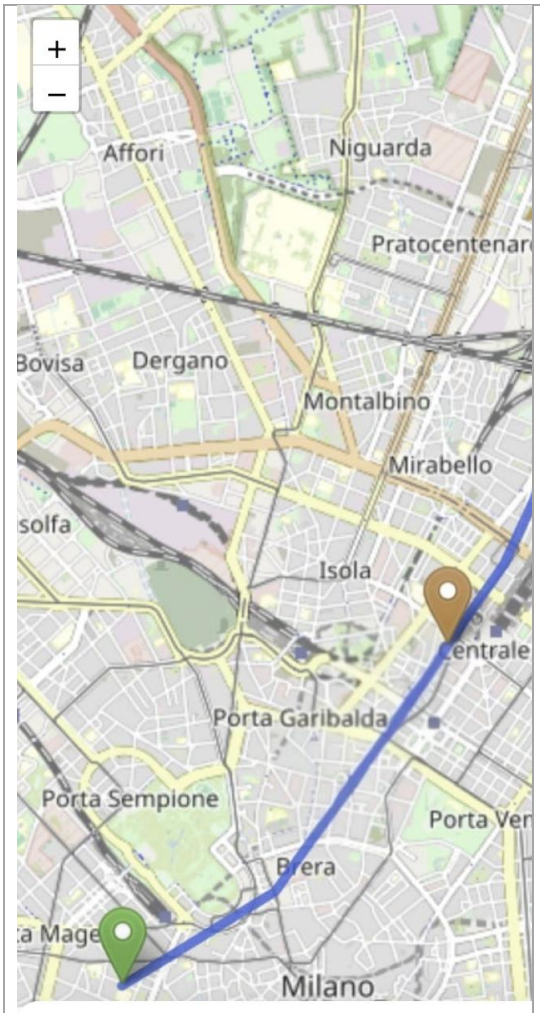


Figure 34 - Expanded map view with street sequence

Tapping the map preview expands it to full-screen, displaying zoom controls (+/-), a close button (X), and the street-by-street sequence below the map showing the ordered list of streets traversed.

3.3. Registered User Interface

Registered users have access to the complete BBP feature set, including trip recording, manual path insertion, activity tracking, and personalized statistics. The registered user interface is designed to support both individual cycling goals and community contribution through an intuitive dashboard and specialized workflows.

3.3.1. Homepage and Dashboard

The homepage serves as the central hub for registered users, providing quick access to main functionalities and displaying personalized activity statistics and recent trips.

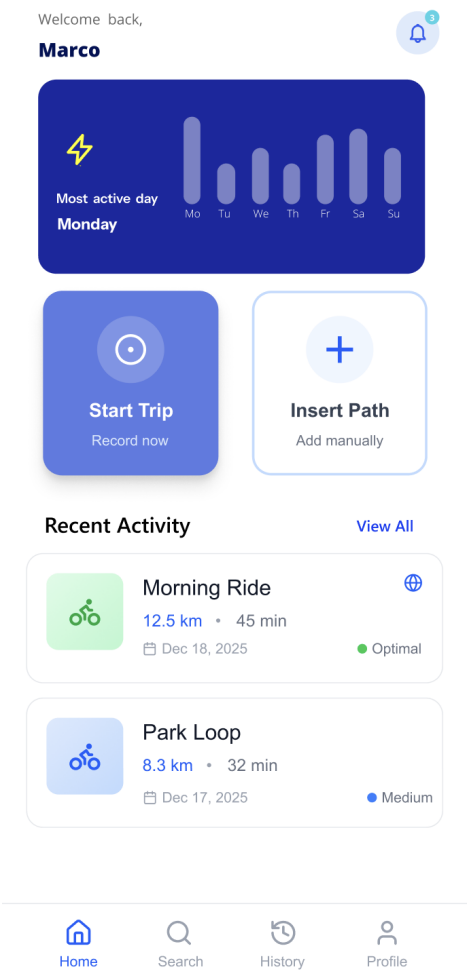


Figure 35 - Registered user homepage

The homepage features:

- Personalized greeting with user's name
- Notification icon in the top-right corner with badge indicator showing the number of unread notifications
- Activity statistics card displaying weekly cycling activity with a bar chart showing the most active day
- Quick action buttons:
  - "Start Trip" for immediate trip recording,
  - "Insert Path" for manual path contribution
- Recent Activity section showing the last completed trips with trip name, distance and duration, date of completion, path status (color-coded as explained in Figure 31), and publicity indicator (globe icon) showing if the trip data has been shared with the community
- Bottom navigation bar for accessing main sections (Home, Search, History, Profile). Active section is highlighted in primary blue color, inactive sections in grey.

3.3.2. Recording Trip Flow

The trip recording feature allows RU to manually start and stop recording cycling activity.

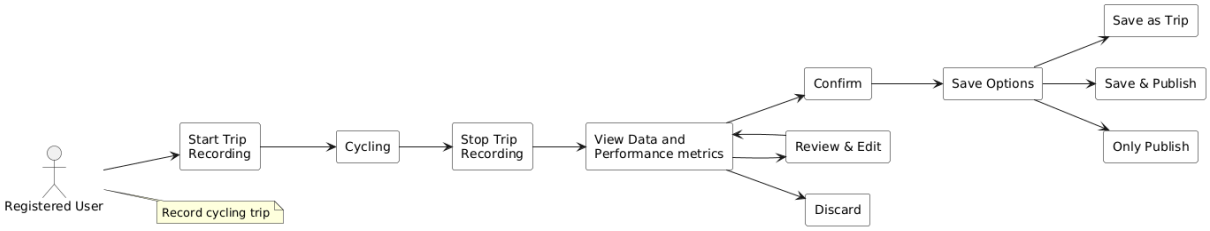


Figure 36 – Trip recording flow

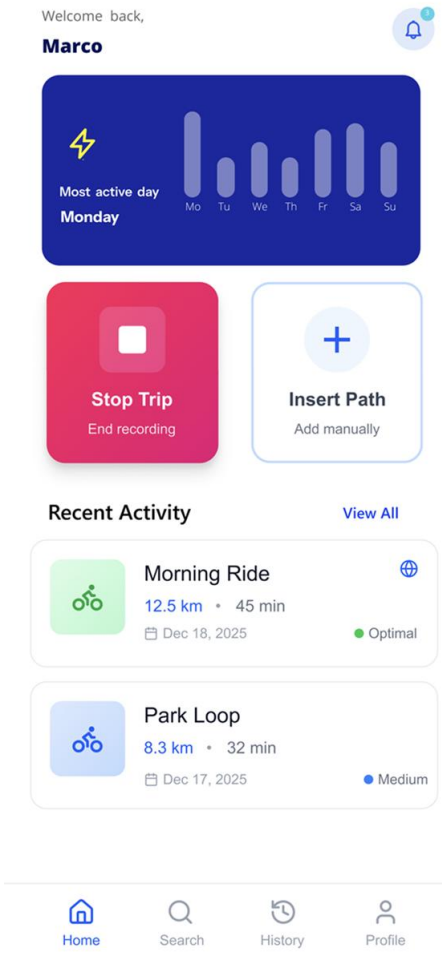


Figure 37 - Active trip recording screen

When the user taps "Start Trip" from the homepage quick actions, the interface enters the active recording state. The quick action button transforms to "Stop Trip - End recording" with a distinctive red background to clearly indicate that recording is in progress. The application operates in the background, automatically tracking GPS position and monitoring motion sensors to detect obstacles (if enabled). The user continues cycling without further interaction until ready to tap "Stop Trip."

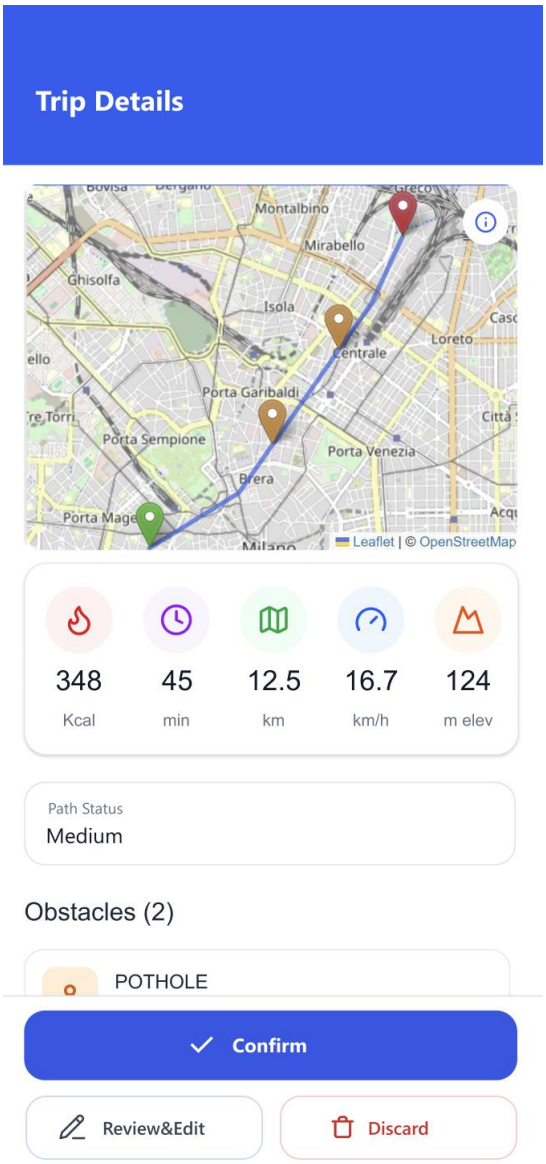


Figure 38 - Trip data and performance metrics data

After tapping "Stop Trip," the user is redirected to the Trip Details screen. A notification is also sent, enabling review of the trip details at a later time if the user chooses to exit. The screen provides a complete summary of collected data and performance metrics. The screen includes a map preview showing the full traced route with green marker for the start point, red marker for the destination, and orange markers for detected obstacles. Below the map, five key statistics are displayed with intuitive icons: calories burned (flame icon), duration in minutes (clock icon), distance in kilometers (map icon), average speed in km/h (speedometer icon), and elevation gain in meters (mountain icon). A path status assessment is shown with color-coded indicator (as explained in Figure 31). Scrolling down reveals the obstacles section with total count and individual entries showing obstacle type and location. At the bottom, three action options are available:

- **Confirm** (proceed to save/publish options)
- **Review & Edit** (modify data)
- **Discard** (delete recording without saving)

Tapping the map expands it to full-screen view (as explained for Figure 34).

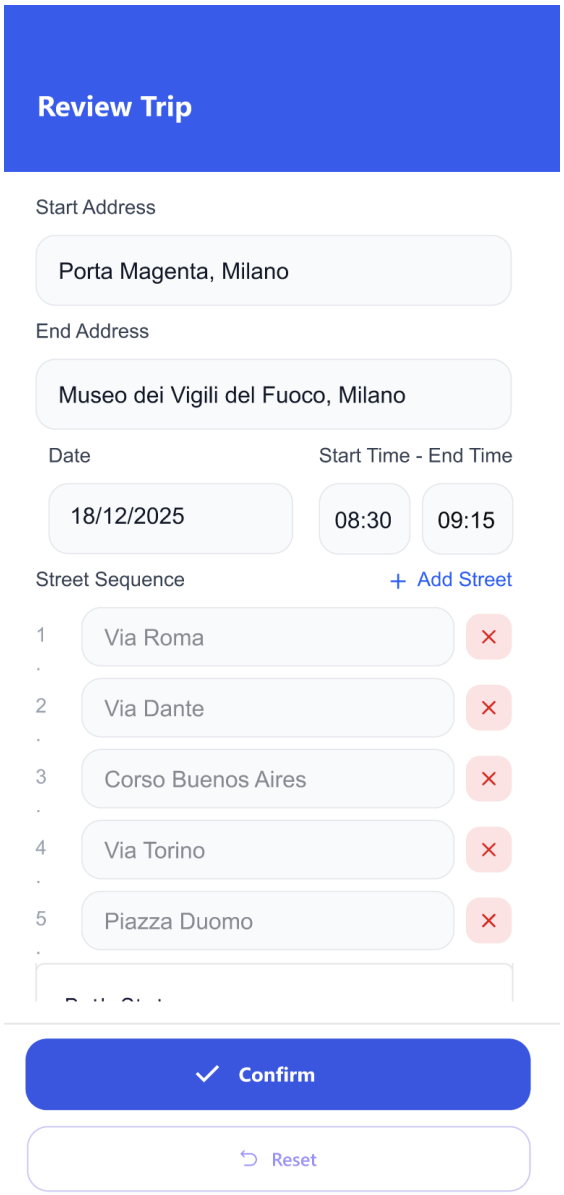


Figure 39 - Review Trip (upper section)

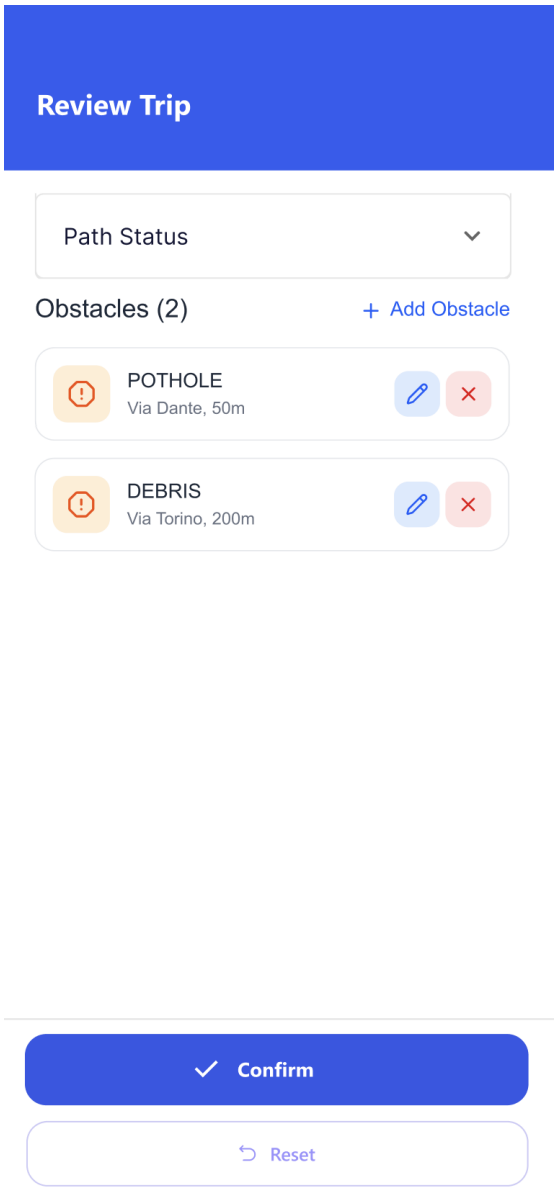


Figure 40 - Review Trip (lower section)

The Review & Edit screen allows users to modify data before final confirmation. Users can modify origin, destination, date, time range, and street sequence with drag-and-drop reordering and add/remove functionality. Path status can be adjusted via dropdown. Scrolling down reveals the obstacles section which displays each obstacle card with type, location, an edit button (pencil icon) for modifications, and a delete button (X) for removal. The "+ Add Obstacle" option allows users to report additional issues. The "Confirm" button saves changes and proceeds to save/publish options, while "Reset" cancels all edits and restores original data.

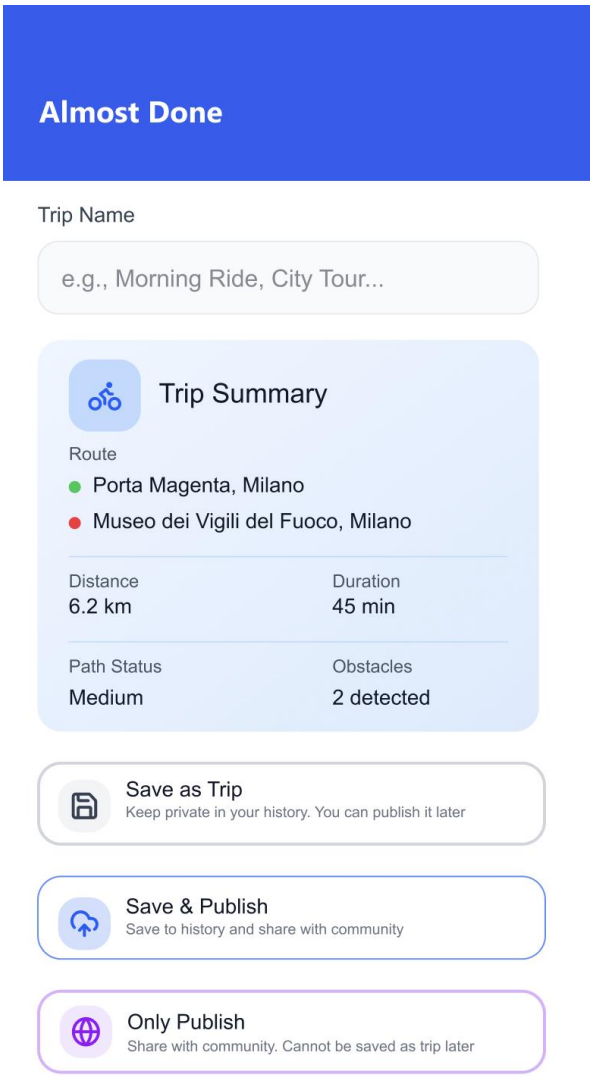


Figure 41 – Save and publication options screen

The "Almost Done" screen allows users to optionally name their trip and choose how to handle the recorded data. A Trip Summary card displays route information.

Three save options are available:

- **Save as Trip:** Keep private in personal history with option to publish later
- **Save & Publish**: Save to history and share with community database
- **Only Publish**: Share with community without saving as personal trip

Each option includes a descriptive subtitle explaining its functionality and implications. After selecting an option, a success confirmation screen displays with a checkmark icon and an appropriate message confirming the action taken.

3.3.3. Manual Path Insertion Flow

RU can manually insert bike path information without recording an active trip.

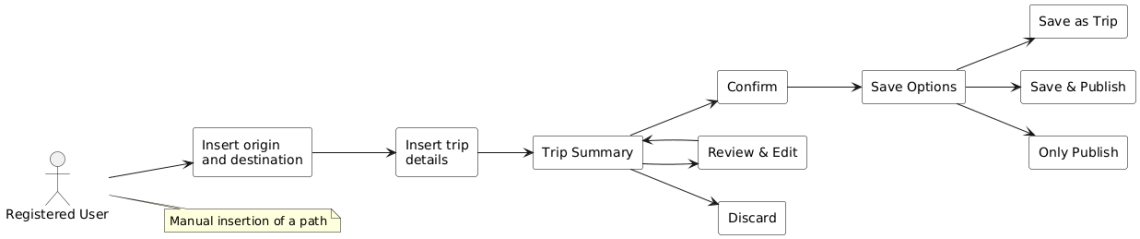


Figure 42 – Manual path insertion flow

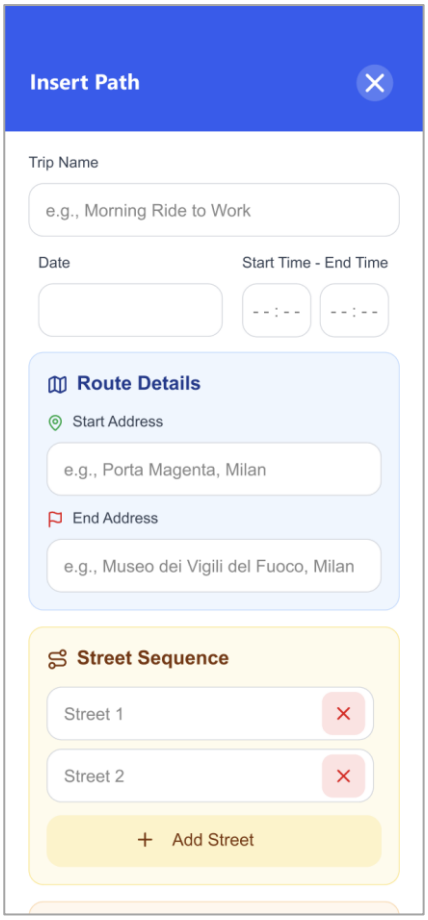


Figure 43- Route definition (upper section)

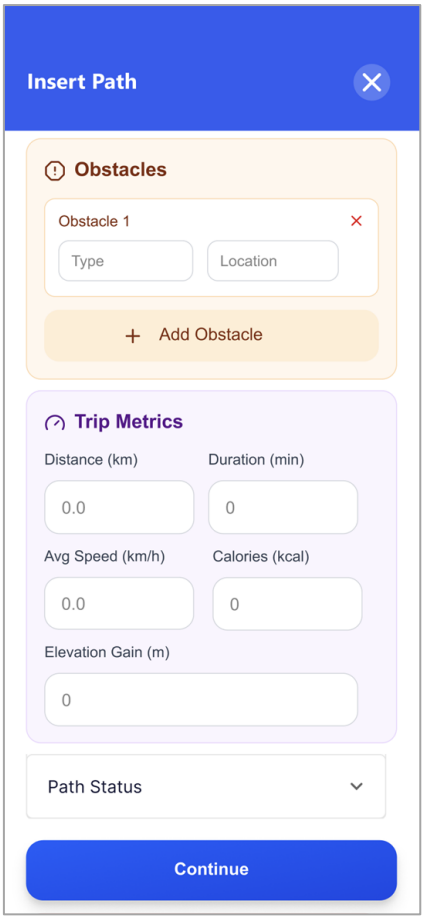


Figure 44 - Route definition (lower section)

Upon selecting "Insert Path" from the homepage, users define route information: trip name, date, time range, start address, end address, and reorderable street sequence with add/remove functionality. Scrolling down users can enter obstacles (type and location fields with add/remove buttons), trip metrics (distance, duration, average speed, calories, elevation gain), and select path status from a dropdown. The "Continue" button proceeds to the Trip Summary screen. The close button (X) discards all data and exits.

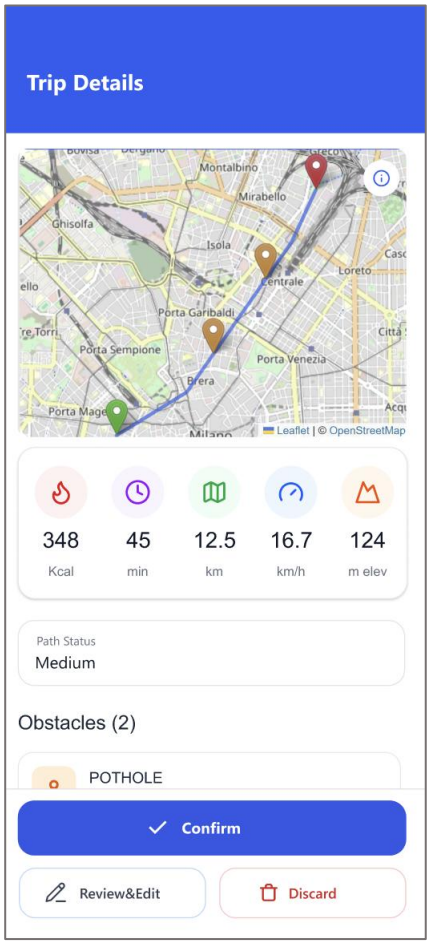


Figure 45- Trip details for manual path insertion

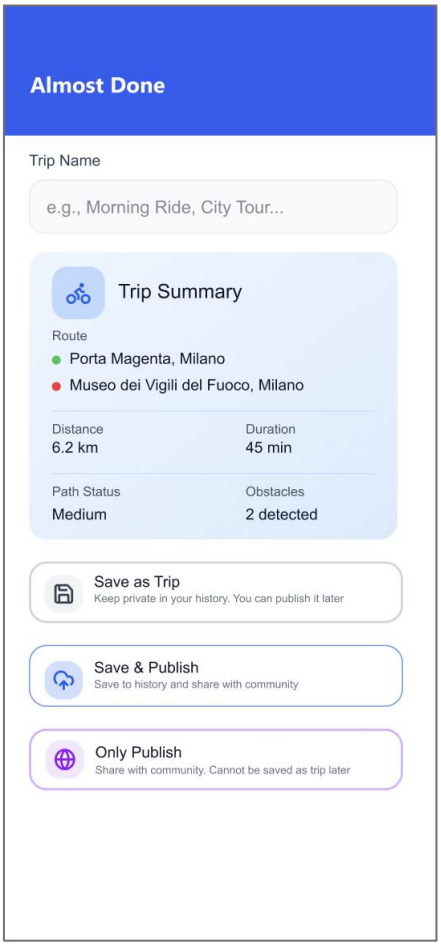


Figure 46- Save and publish options for manual path insertion

The Trip Details screen follows the same layout in the Figure 37, displaying the complete path information with map visualization, all performance metrics, path status and scrolling down the list of obstacles. The key difference is that the trip name is already populated from the initial form entry, rather than being entered later.

Users have the same three options:

- **Confirm**: Proceeds to save and publish options
- **Review & Edit**: Returns to the previous form screens to modify information
- **Discard**: Cancels the insertion without saving

Upon selecting Confirm, the user is redirected to the "Almost done" screen, where users choose how to handle the manually inserted path with the same layout as Figure 42 with the three save options (Save as Trip, Save & Publish, Only Publish) but the trip name field is not shown as it was already defined in the initial form.

3.3.4. Automated Trip Detection Flow

When auto-detection mode is enabled in settings, the system automatically detects cycling activity and records trip data without requiring manual start/stop actions.

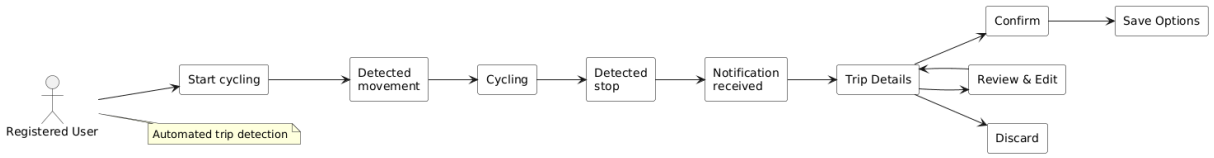


Figure 47 – Automated trip detection flow

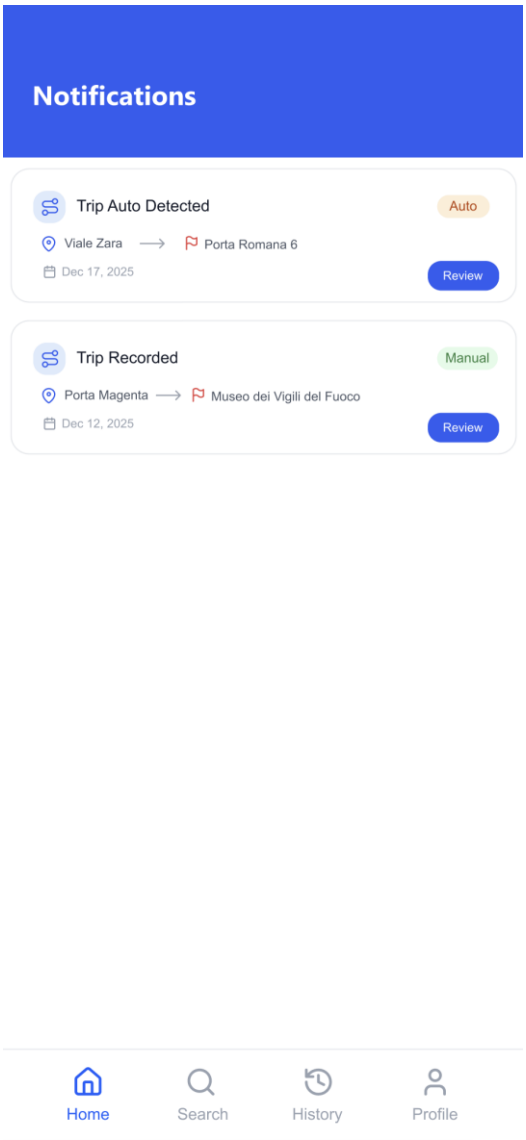


Figure 48 – Notification screen

When the system detects that the user has started cycling, the trip is automatically recorded. Once the user stops for a certain period, they receive a notification labeled "Trip Auto Detected" with an "Auto" badge. The notification card displays:

- Origin and destination addresses
- Trip date
- "Review" button to access trip details.

Tapping the "Review" button opens the Trip Details screen (same layout as Figure 39) with automatically collected data including map visualization, performance metrics, path status, and detected obstacles. From here, users follow the same workflow as manual trip recording: Confirm to proceed to save options, Review & Edit to modify data, or Discard to delete the recording.

It should be noted that in manual trip recording mode, the user also receives a notification labeled "Trip Recorded" with a "Manual" badge if they stop the recording, allowing them to review the trip later if they are not available to do so immediately.

3.3.5. Search Bike Paths

Registered users have access to the same path search functionality as guest users, with an enhanced interface that reflects their authenticated status.



Figure 49 – Registered user search flow

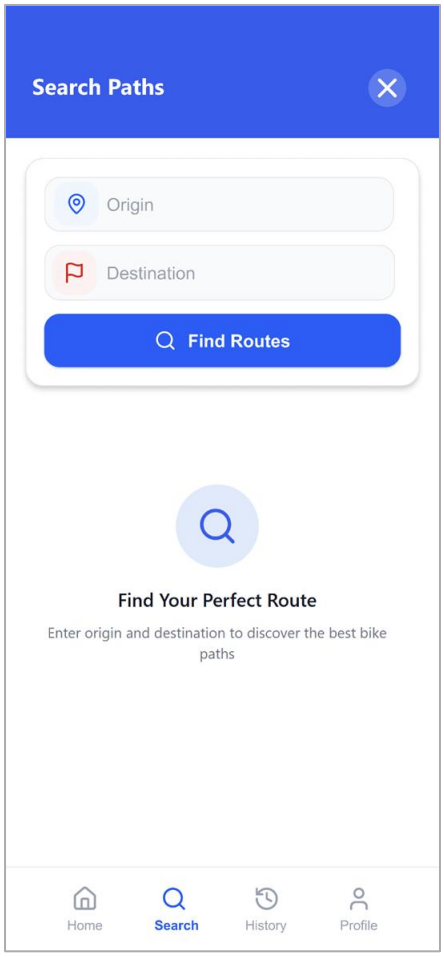


Figure 50 - Search interface

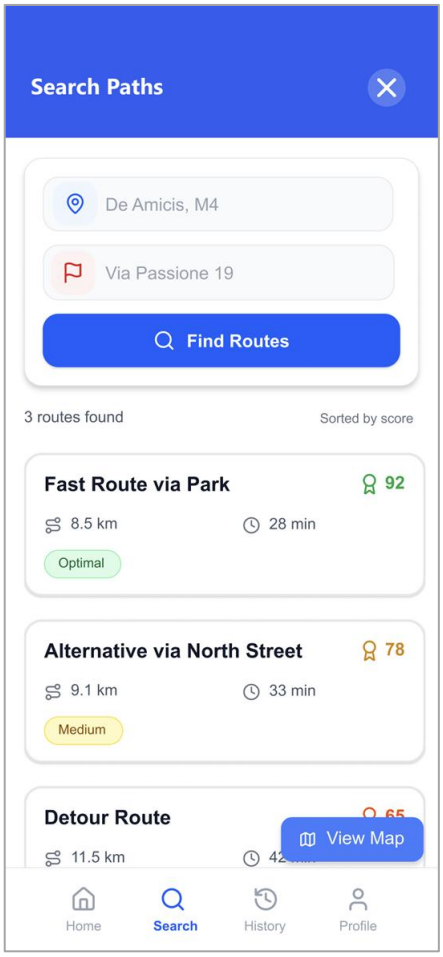


Figure 51 - Ranked search results

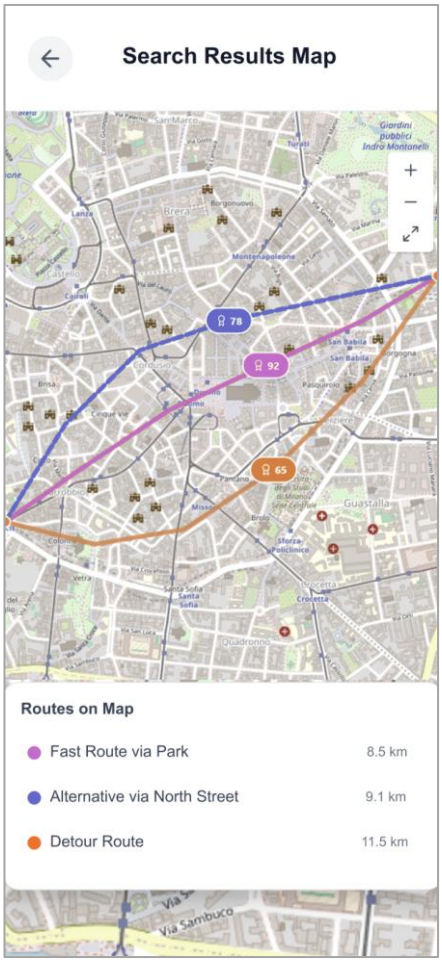


Figure 52 - Map visualization

Same flow and layout as Figures 30-32, with the only differences being: no "Guest Mode" badge, no registration icon, added close button (X), and navigation bar present.

3.3.6. My Trips and Weather Enrichment

The My Trips section provides users with access to their complete cycling history, including statistics and detailed trip information. When available, trip details are enriched with historical weather data from the time and location of the recorded activity.

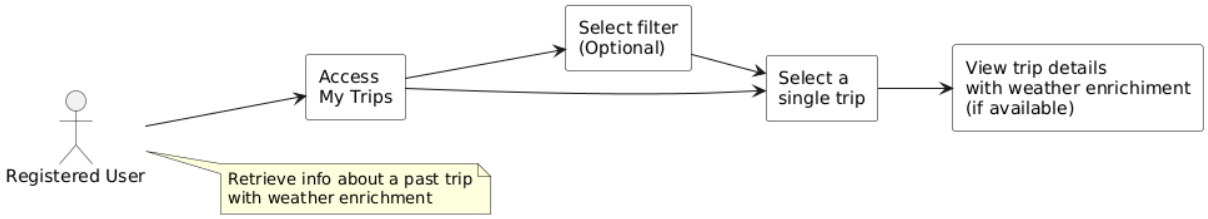


Figure 53 - My Trips access flow

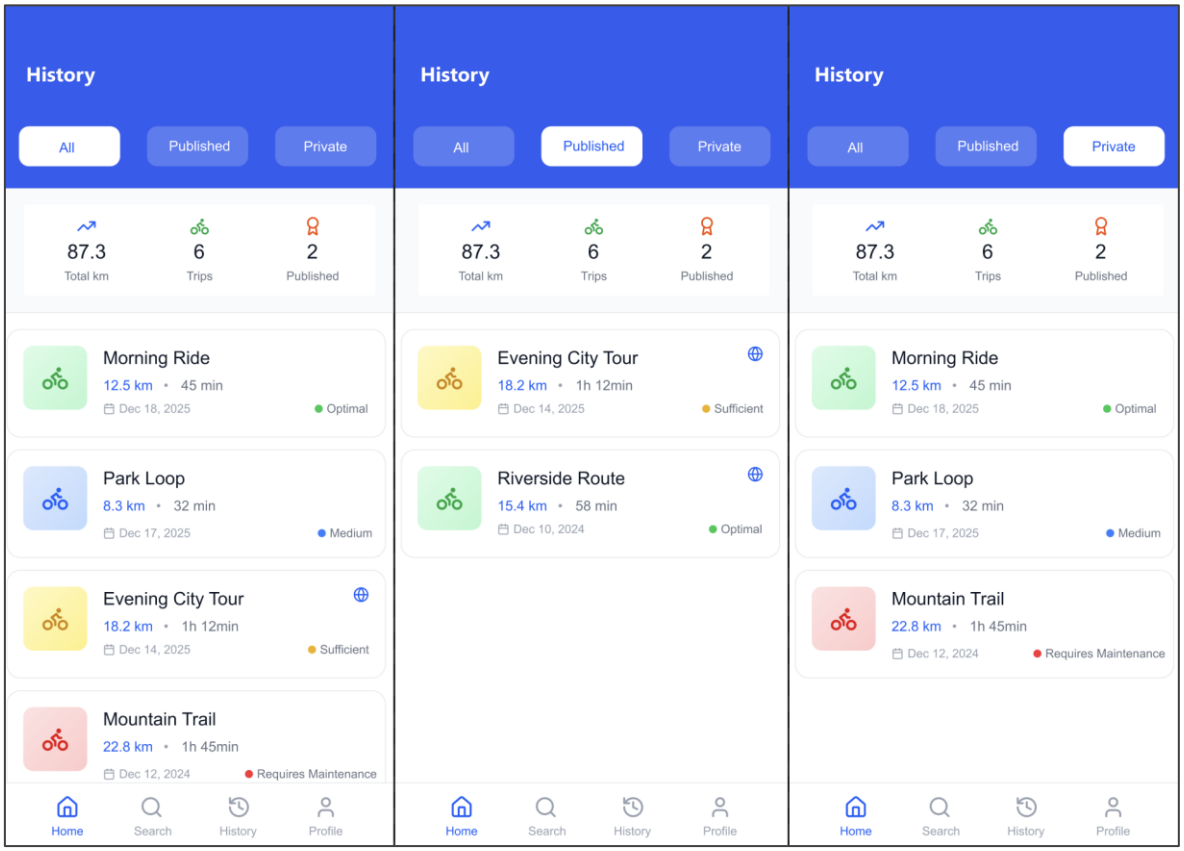


Figure 54 - Trip history with filtering options

The History screen features three filter tabs at the top: All (shows all trips), Published (displays only trips shared with the community), Private (shows trips saved privately without community sharing). Below the tabs, aggregate statistics display total kilometers cycled, number of trips, and number of published paths. Each trip appears as a card with a color-coded icon based on path status (green for Optimal, blue for Medium, yellow for Sufficient, red for Requires Maintenance -

same color coding as explained in Figure 31), trip name, distance, duration, date, and path status badge. Published trips are marked with a globe icon in the top-right corner of their card. Users can optionally apply filters or directly select any trip to view detailed information.

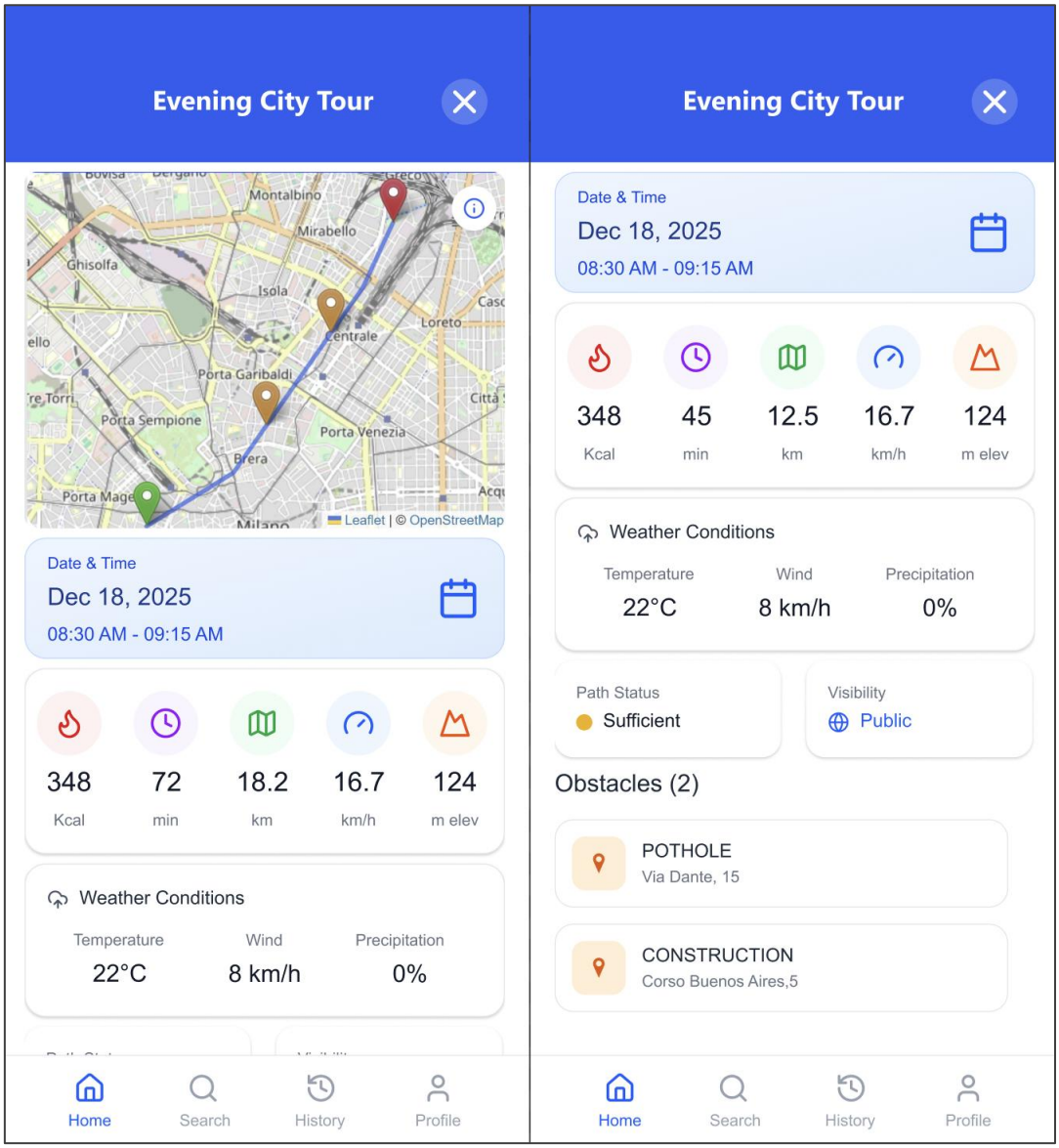


Figure 55 - Trip details with weather enrichment

Selecting a trip opens the detailed view showing the route map with markers (green start, red destination, orange obstacles), date and time information, and performance metrics (calories, duration, distance, average speed, elevation gain). For trips with available weather data, a "Weather Conditions" section displays historical weather information for the trip's timestamp and location: Temperature (°C) , wind speed (km/h), precipitation (%). The map preview is tappable to expand to full-screen view as explained for Figure 34.

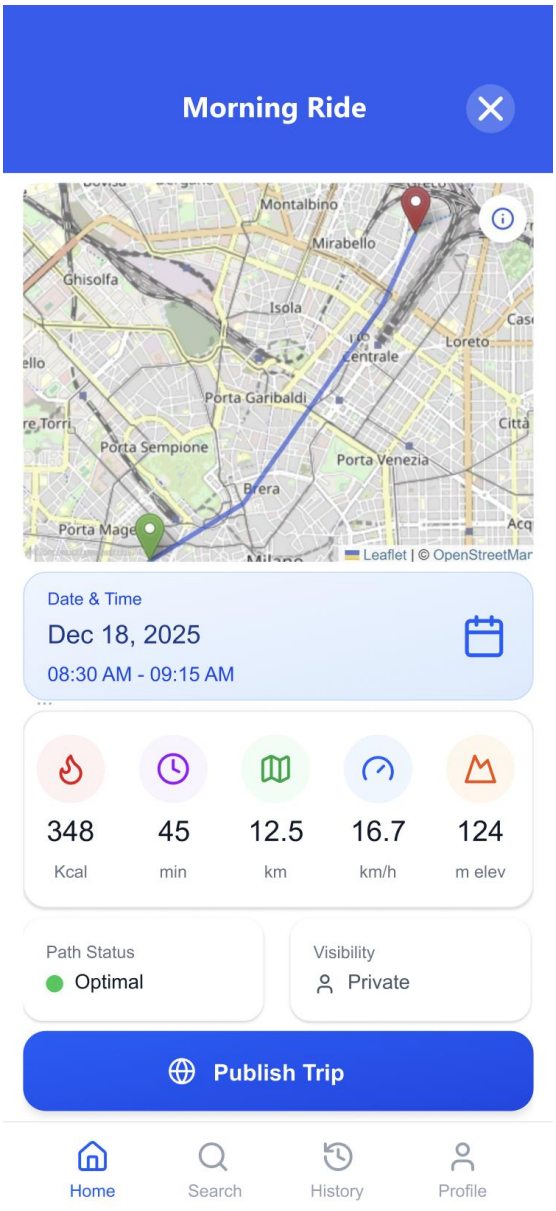


Figure 56 - Private trip with publish option

For private trips, users can choose to publish them to the community using the "Publish Trip" button at the bottom of the screen. This allows users to initially save trips privately and later decide to share them with the community.

Note that the weather enrichment section may not be present for some trips if weather data could not be retrieved for some reason during the saving process.

3.3.7. User Profile and Settings

The Profile section allows users to view their account information, manage settings, and access application preferences.

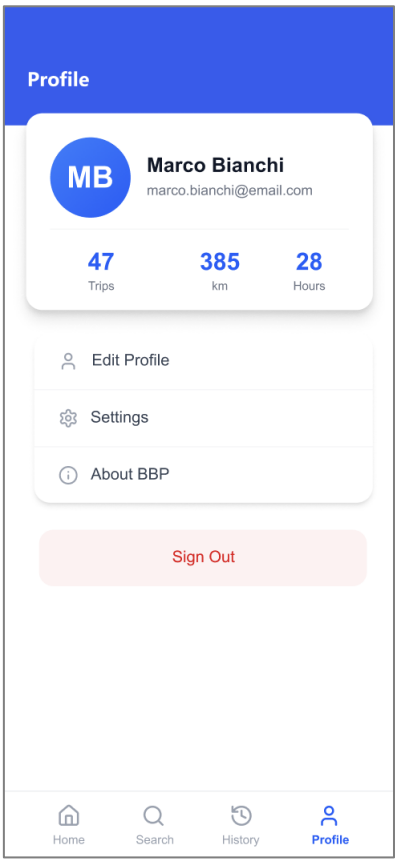


Figure 57 - User profile

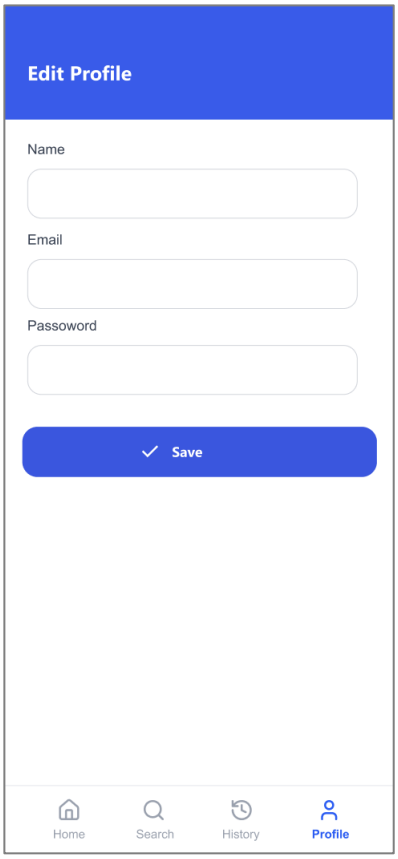


Figure 59 - Edit profile

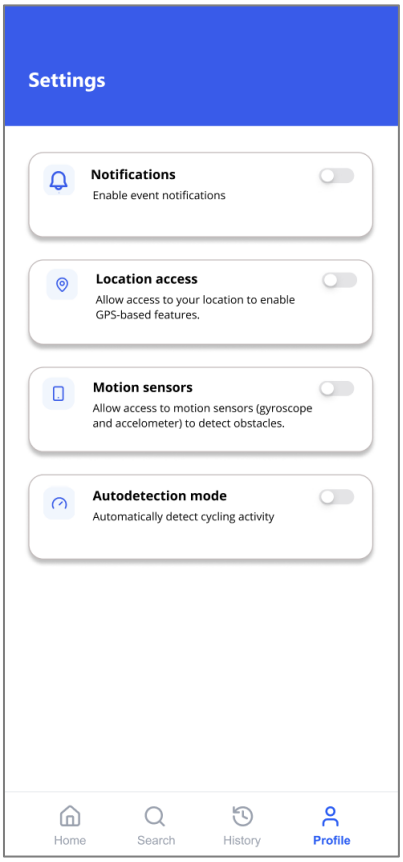


Figure 58 - Settings

The Profile screen displays user information (avatar with initials, name, email), aggregate statistics (trips, kilometers, hours), and navigation options for Edit Profile, Settings, and About BBP, with a Sign Out button at the bottom.

The Edit Profile screen allows users to update their Name, Email, and Password with a Save button to commit changes.

The Settings screen provides toggle switches for the same permissions configured during registration (Figure 25): Notifications, Location access (for GPS features), Motion sensors (for obstacle detection), and Autodetection mode (for automatic cycling activity detection).

4. Requirements traceability

1. Dashboard Manager

- **R03** - BBP displays bike paths on a map
- **R04** - BBP displays a list of paths ranked by their score when multiple paths are available
- **R05** - BBP displays the score for each bike path
- **R15** - BBP provides statistics for each recorded trip including total distance, average speed, and performance metrics
- **R16** - BBP maintains a historical record of all trips recorded
- **R27** - BBP displays collected data with all the suggestions assigned to RU for review

2. Authentication Manager

- **R01** - BBP enables users to register an account
- **R06** - BBP allows RU to log in
- **R07** - BBP allows RU to log out

3. Profile Manager

- **R08** - BBP enables RU to view and edit their profile information
- **R09** - BBP enables RU to grant GPS access to the system
- **R10** - BBP enables RU to grant access to accelerometer and gyroscope sensors
- **R11** - BBP enables RU to enable/disable automated data collection mode

4. Searching Manager

- **R02** - BBP enables users to search a bike path between an origin and a destination
- **R37** - BBP makes published bike paths searchable and visible to all users

5. Manual Insertion Manager

- **R17** - BBP enables RU to publish an existing trip as a bike path
- **R18** - BBP allows RU to manually insert bike path information by specifying street addresses
- **R19** - BBP enables RU to assign a path status using predefined categories
- **R20** - BBP allows RU to report the presence of relevant obstacles along paths
- **R26** - BBP suggests path information (origin, destination, street names, status) based on collected data
- **R28** - BBP allows RU to review and correct all the suggested data gathered
- **R29** - BBP allows RU to reject or correct each obstacles detected
- **R30** - BBP allows RU to confirm collected data information
- **R31** - BBP enables RU to save path information as a personal trip

- **R32** - BBP enables RU to publish path information as a bike path to share with the community
- **R33** - BBP enables RU to discard path information without saving or publishing
- **R36** - BBP computes a path score based on path status and obstacles

6. Collector Manager

- **R12** - BBP allows RU to start/stop recording of their cycling trips for personal activity tracking
- **R13** - BBP collects GPS data during manual trip recording
- **R14** - BBP collects accelerometer and gyroscope data during manual trip recording when sensors are enabled
- **R21** - BBP automatically detects when the user is cycling based on movement speed
- **R22** - BBP collects GPS data from the mobile device to reconstruct the followed path
- **R23** - BBP acquires data from the mobile device's accelerometer and gyroscope when enabled to detect obstacles
- **R24** - BBP automatically stops data collection when cycling activity is no longer detected
- **R25** - BBP processes collected GPS and sensor data to calculate trip statistics

7. Weather Integration Manager

- **R34** - BBP retrieves meteorological data from external service when saving a trip, using timestamp and GPS data
- **R35** - BBP enriches trip with weather information (conditions, temperature, wind speed) if available

8. Database Query Manager

- **R16** - BBP maintains a historical record of all trips recorded
- **R31** - BBP enables RU to save path information as a personal trip
- **R32** - BBP enables RU to publish path information as a bike path
- **R37** - BBP makes published bike paths searchable and visible to all users

5. Implementation, integration and test plan

5.1. Overview

In this chapter, we will discuss the implementation of the BBP mobile application and backend system, the integration strategy, and the testing approach that will be used to ensure the quality and reliability of the software.

The methodology used to implement the BBP system is the Bottom-Up approach, which is a software development strategy that starts by implementing the lower-level modules first, and then progressively integrating them to create higher-level modules. By using this approach, we can ensure that the foundational modules are working correctly before integrating them into more complex modules. Each module will require a Driver to test its functionality. The Driver will simulate the behaviour of higher-level modules and will call the lower-level module interfaces to test their functionality.

5.2. Features Identification

The BBP system is a comprehensive mobile application with different features used by Guest Users (GU) and Registered Users (RU). The features are identified as follows:

Guest User Features

This set of features is available to users who have not created an account. Guest users have limited functionality but can still benefit from the community-contributed bike path information.

- **[GU.1] Continue as Guest**: This feature allows users to browse the application without creating an account, accessing read-only features.
- **[GU.2] Search Bike Paths**: This feature allows guest users to search for bike paths between specified origin and destination points.
- **[GU.3] View Paths on Map**: This feature allows guest users to visualize bike paths on an integrated map with status information and obstacle indicators.
- **[GU.4] Register Account**: This feature allows guest users to create a personal account to access full BBP functionality.

Registered User Features

This set of features is used by RUs to manage their trips, contribute path information, and access personalized statistics.

- **[RU.1] Authentication Management**: This feature allows registered users to log in using their credentials, reset passwords, and manage their authentication status.
- **[RU.2] Trip Recording**: This feature allows users to record cycling trips with GPS tracking and optional sensor data collection for obstacle detection.
- **[RU.3] Manual Path Insertion**: This feature allows users to manually insert bike path information by specifying street names and path status.
- **[RU.4] Automatic path detection**: This feature allows users to automatically detect when a user is cycling and collect GPS data and optional sensor data collection for obstacle detection.
- **[RU.5] Path Information Review**: This feature allows users to review, confirm, or correct automatically collected path information before saving or publishing.
- **[RU.6] Trip Storage and Statistics**: This feature allows users to save trips privately to "My Trips" and view performance metrics including distance, average speed, and duration.
- **[RU.7] Path Publication**: This feature allows users to publish path information as a BikePath to share with the community.
- **[RU.8] Profile Management**: This feature allows users to view and edit their profile information.
- **[RU.9] Weather Data Enrichment**: This feature allows the system to automatically enrich trip records with meteorological data when available.

Implementation Feature Categories

Based on the architectural design presented in Chapter 2, the features are further organized into implementation categories:

[F1] Authentication Features:

- [RU.1] Authentication Management: Login, password reset, and session management
- [GU.4] Register Account: New user registration with email verification

[F2] Trip Recording Features:

- [RU.2] Trip Recording: GPS tracking and sensor data collection

- [RU.8] Weather Data Enrichment: Retrieval and association of meteorological data
- [RU.4] Path Information Review: Confirmation and correction interface for collected data
- PATH AUTOMATIC FEATURES

[F3] Path Management Features:

- [RU.3] Manual Path Insertion: Manual entry of bike path information
- [RU.5] Trip Storage and Statistics: Saving trips and calculating performance metrics
- [RU.6] Path Publication: Publishing paths to the community database

[F4] Path Query Features:

- [GU.2] Search Bike Paths: Route search between origin and destination
- [GU.3] View Paths on Map: Map visualization with path scoring
- Path scoring computation based on status and effectiveness

[F5] Profile Features:

- [RU.7] Profile Management: User profile viewing and editing

[F6] Offline Support Features:

- Local trip storage during offline mode
- Automatic synchronization when connectivity is restored

5.3. Integration Strategy

The integration strategy will start as soon as the DBMS and application server infrastructure are built and ready to be used. The integration will be done using the Bottom-Up approach, beginning with the foundational data access layer.

Step 0: Database and Query Manager

The first module to be implemented and tested will be the **Database Query Manager** module, which handles all database operations and returns the fundamental classes that are going to be used by other modules. The Query Manager module contains and generates objects of the classes that are part of the Model in the three-tier architecture. The Database Query Manager module will be tested using a proper Driver that will simulate the behaviour of the other modules that will use the Query Manager module.

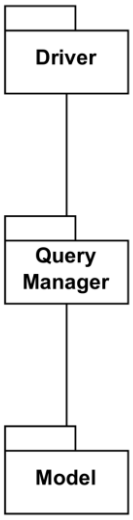


Figure 60 - Integration Strategy, Step 0

Step 1: Notification Manager and Weather Integration Manager

The next modules that will be implemented and tested will be the Notification Manager and the Weather Integration Manager. These two modules are independent of each other and can be implemented and tested simultaneously. The Notification Manager handles both email notifications through the Mail Server for account verification and password recovery, and push notifications to mobile devices for real-time alerts and updates. The Weather Integration Manager retrieves meteorological data from the external Weather Server to enrich trip records when available. Both modules communicate only with their respective external services and do not depend on the Database Query Manager.

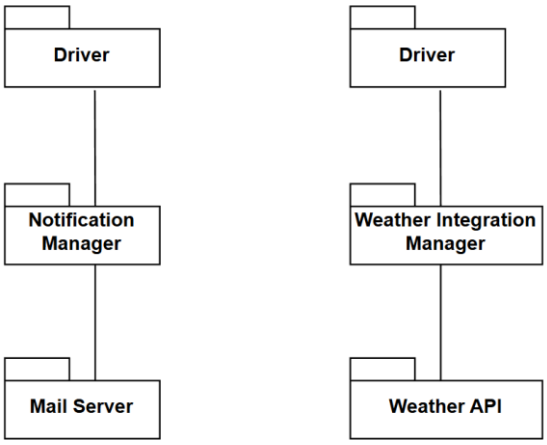


Figure 61 - Integration Strategy, Step 1

Step 2: Authentication Manager

The next module to be implemented and tested will be the Authentication Manager. This module depends on the Database Query Manager for user credential storage and retrieval and on the Notification Manager to send verification emails during registration and password recovery. It handles user registration, login, logout, password hashing, and session token management. The Authentication Manager will be tested using a Driver that simulates client authentication requests.

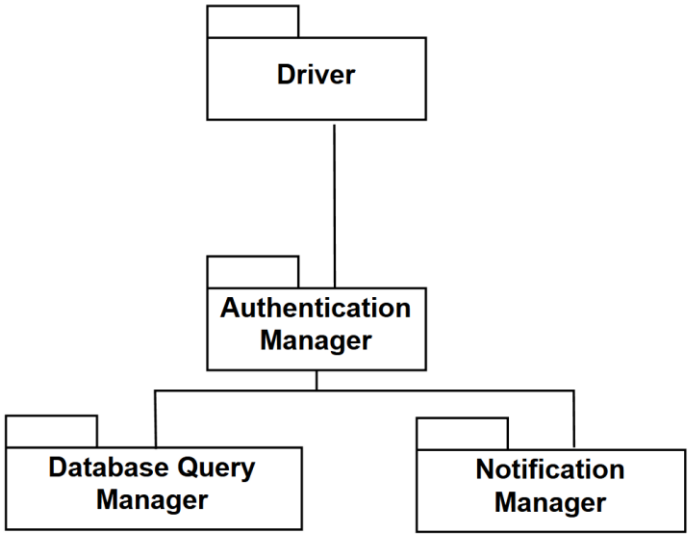


Figure 62 - Integration Strategy, Step 2

Step 3: Profile Manager

The Profile Manager module will be implemented and tested next. This module manages user profile information and device permission settings (GPS, accelerometer, gyroscope, automated detection). It depends on the Database Query Manager for storing and retrieving user preferences. The module will be tested with a Driver simulating profile viewing and editing operations.

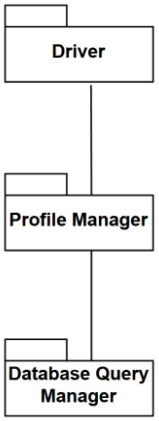


Figure 63 - Integration Strategy, Step 3

Step 4: Collector Manager

The next module that will be implemented and tested will be the Collector Manager. This module collects GPS coordinates and sensor data (accelerometer and gyroscope) during trip recording. The Collector Manager stores all collected data temporarily in memory during the recording session and does not persist data to the database directly. A Driver will simulate trip start, data collection, stop, and review workflows.

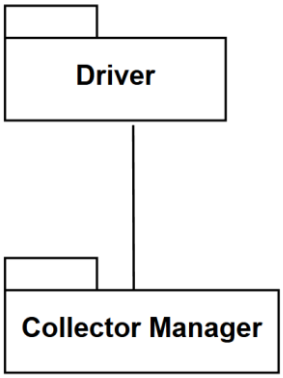


Figure 64 - Integration Strategy, Step 4

Internally, the Collector Manager coordinates three specialized sub-components that can be developed and tested independently: the **GPS Handler** manages GPS tracking and route reconstruction, the **Sensor Data Collector** processes accelerometer and gyroscope data for obstacle detection, and the **Statistics Calculator** computes trip metrics from the collected data. These sub-components work together during trip recording, with each having dedicated unit testing before integration into the complete Collector Manager module.

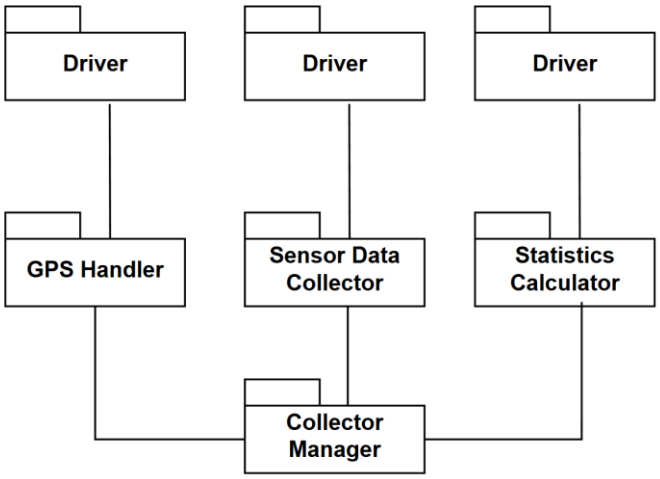


Figure 65 - Collector Manager - Internal Sub-components Testing

Step 5: Manual Insertion Manager

The Manual Insertion Manager module will be implemented next. This module handles both automated path detection and manual path insertion workflows. It depends on the Database Query Manager for persisting trips and published bike paths. When users save trips, the Manual Insertion Manager calls the Weather Integration Manager to enrich trip data with meteorological information. The module also integrates with the external Maps API for geocoding and address validation during manual path insertion. A Driver will test the complete data processing pipeline including manual street insertion with validation, automated detection with obstacle confirmation, trip saving with weather enrichment, and path publication with score calculation.

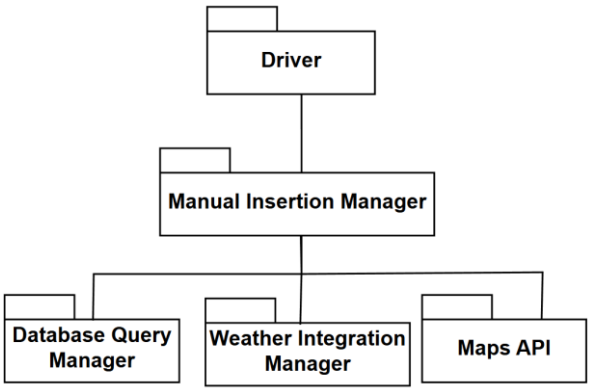


Figure 66 - Integration Strategy, Step 5

Step 6: Searching Manager

The next module that will be implemented and tested will be the Searching Manager. This module processes bike path search queries between origin and destination points. It depends on the Database Query Manager for querying published bike paths. The module also integrates with the external Maps API for geocoding user-entered addresses and preparing paths for map visualization. A Driver will test search functionality with various origin-destination combinations, path ranking by score, and map visualization preparation.

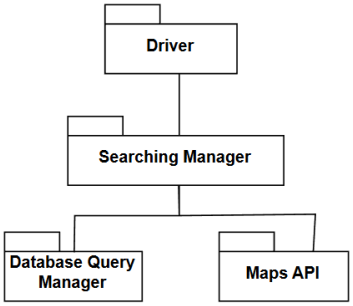


Figure 67 - Integration Strategy, Step 6

Step 7: Dashboard Manager

The last step of integration will be the implementation and testing of the Dashboard Manager module. The Dashboard Manager coordinates all user interactions through the mobile interface and routes requests to the appropriate internal components. It manages the application navigation flow between different features (registration/login, recording, path insertion, search, trip history, profile management) and ensures proper synchronization between the mobile app and server components. This final integration step also includes the complete integration of the mobile presentation layer.

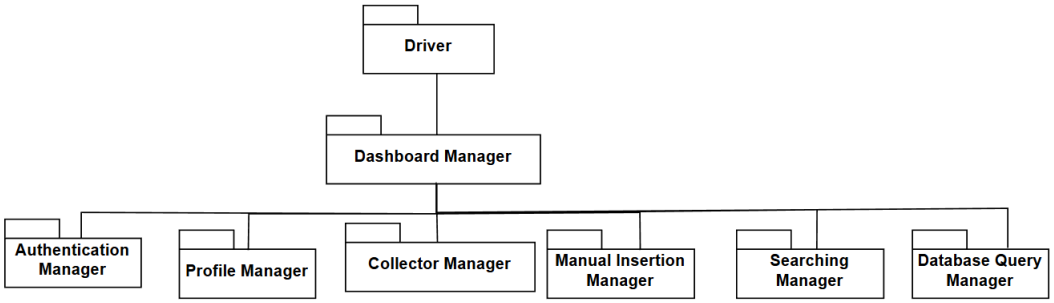


Figure 68 - Integration Strategy, Step 7

5.4. System Testing Strategy

The testing strategy for BBP will be comprehensive and multifaceted, ensuring the system meets all functional and non-functional requirements specified in the RASD. The following testing approaches will be detailed and conducted:

1. Functional Testing

The system will be tested to ensure that all workflows operate as intended and meet the specified functional requirements, satisfying user needs. This includes:

- **Authentication Testing**: Verify registration, login, password reset, and session management
- **Automatic detection testing**
- **Trip Recording Testing**: Validate GPS tracking accuracy, sensor data collection, and automatic cycling detection
- **Path Management Testing**: Test manual insertion, automated data collection, and path publication workflows
- **Route Query Testing**: Verify search functionality, path scoring accuracy, and map visualization

- **Offline Functionality Testing**: Ensure data collection works offline and synchronizes correctly when online

2. Performance Testing

The application will undergo tests to confirm it can handle expected user loads, sustain peak conditions, and maintain response times within acceptable latency. This includes:

- **Load Testing**: Test system behaviour with multiple concurrent users recording trips
- **Response Time Testing**: Ensure route queries return results within 2-3 seconds
- **Database Performance**: Validate efficient spatial queries on geographic data
- **Mobile Performance**: Test app responsiveness and battery consumption during trip recording
- **API Latency**: Measure and optimize backend API response times

3. Security Testing

The system will be tested to verify that sensitive data is protected, and the application is resilient against unauthorized access and common security threats. This includes:

- **Authentication Security**: Test password hashing, token-based authentication, and session management
- **HTTPS Encryption**: Verify all client-server communication is encrypted
- **Data Privacy**: Ensure personal trip data is only accessible to the owner
- **Input Validation**: Test against SQL injection, XSS, and other common vulnerabilities
- **Credential Storage**: Verify secure storage of passwords and authentication tokens

4. Integration Testing

Tests will verify that all system components work correctly together and external service integrations function as expected. This includes:

- **Maps Service Integration**: Test GPS coordinate processing and route visualization
- **Weather Service Integration**: Verify meteorological data retrieval and association

- **Mail Service Integration**: Test email verification and notification delivery
- **Native Sensor Integration**: Validate accelerometer and gyroscope data collection

5. Usability Testing

The system will be tested to ensure the mobile interface is intuitive, accessible, and provides a positive user experience. This includes:

- **Navigation Testing**: Verify users can easily access all features
- **Trip Recording UX**: Ensure automatic tracking works seamlessly without user intervention
- **Data Confirmation Workflow**: Test the review and correction interface for collected data
- **Map Visualization**: Verify path rendering is clear and informative
- **Error Handling**: Ensure error messages are helpful and non-technical

6. Failure Testing

Tests will simulate unexpected scenarios, such as network failures, GPS signal loss, or device sensor malfunctions, to confirm the system can recover gracefully and maintain reliability with minimal disruption. This includes:

- **Network Loss Recovery**: Test offline mode and automatic synchronization
- **GPS Signal Loss**: Verify graceful handling when location services are unavailable
- **Sensor Failure**: Test fallback behaviour when accelerometer/gyroscope is unavailable
- **API Failure**: Verify system resilience when external services (weather, maps, mail) are down
- **Database Failure**: Test error handling and recovery mechanisms

7. Endurance Testing

The system will be tested under prolonged usage conditions to ensure stability, consistent performance, and the absence of issues like memory leaks or resource exhaustion. This includes:

- **Long Trip Recording**: Test continuous GPS tracking for extended periods (2-4 hours)
- **Memory Management**: Monitor mobile app memory usage during extended sessions

- **Background Processing**: Verify trip recording continues correctly in background mode
- **Data Synchronization**: Test synchronization of large datasets after extended offline periods

8. User Acceptance Testing

End-users (cyclists from the biker's association) will validate the application by executing real-world tasks to ensure it meets their requirements and is ready for deployment in production environments. This includes:

- **Real-World Cycling Tests**: Users record actual cycling trips with varying conditions
- **Path Quality Validation**: Verify accuracy of obstacle detection and path status assessment
- **Route Recommendation Testing**: Users evaluate usefulness of suggested bike paths
- **Community Contribution**: Test the complete workflow from trip recording to path publication
- **Feedback Collection**: Gather user feedback on interface, functionality, and overall experience

6. Effort spent

Student	Section	Hours
Anna Cordioli	*1*	3
	2	4
	3	6
	4	1
	5	4
TOTAL		18
Tasnim Elkoush	*1*	2
	2	4
	3	13
	4	0
	5	3
TOTAL		22

7. References

7.1. References

• ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.

7.2. Used Tools

- GitHub - Used for project versioning and sharing.
- Word - Used as editor for writing the document.
- https://online.visual-paradigm.com/.
- plantuml-editor.kkeisuke.dev- Used for sequence diagrams' design.
- Figma - Used for the mock-ups' design.